Smart Traffic System using FPGA Basys3

Prepared by:

Mostafa Mohamed Abdelaziz 58-22509

Ahmed Adel Sanad 58-1051

Ahmed Sherif Elafandy 58-5317

Mohammad Soufi 58-13698

Hassan Elgohary 58-1047

Marwan Elsayed Ali 58-20308

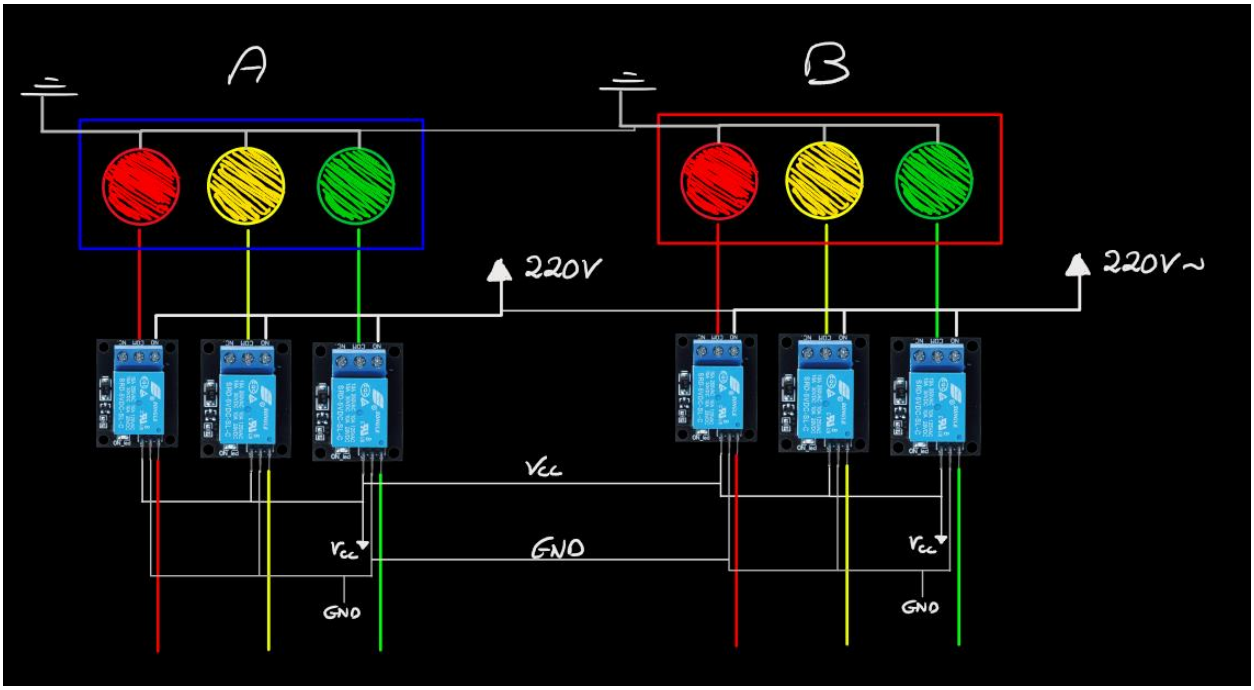
date of submission: 9 December 2024

Introduction:

This project is a Smart Traffic Management System developed using VHDL and implemented on a Basys 3 FPGA board. The purpose of this project is to optimize traffic flow and enhance safety by integrating advanced monitoring and control features. Our System follow the German Traffic System .It includes four main vehicular traffic lights (red, yellow, green) and four pedestrian-specific lights to regulate movement and at the intersection of the lanes it form a square composed of four Ultrasonics . Ultrasonic sensors are positioned centrally to detect vehicle presence and movement, enabling dynamic control of traffic signals. A buzzer system is incorporated to alert for traffic violations, such as vehicles crossing during a red light, promoting safety and rule compliance. Additionally, a DHT11 temperature sensor monitors environmental conditions, offering potential adaptability for extreme weather scenarios. By leveraging the Basys 3 FPGA board and VHDL programming, this project demonstrates the integration of real-time sensing and hardware-based control to address modern urban traffic challenges efficiently.

Project Components and Design :

1)Traffic Light System:



The Traffic Light system is designed to efficiently manage the flow of vehicles and pedestrians while ensuring safety and smooth operation at the intersection.

Overview:

The traffic system operates in **four phases**, each consisting of vehicle and pedestrian traffic signals:

1. **Phase 1**:
    - Two traffic signals for vehicles, each with three LEDs (Red, Yellow, Green) for opposite lanes.
    - Two pedestrian signals, each with two LEDs (Red, Green).

2. **Phase 2**:
    - Similar to Phase 1 but operates in the opposite direction.
3. **Phase 3**:
    - Changes all vehicle signals from Phase 1 and Phase 2 to red.
    - Activates two LEDs (Red, Green) for left-turn traffic.
4. **Phase 4**:
    - Also turns all Phase 1 and Phase 2 vehicle signals red.
    - Activates two LEDs (Red, Green) for left-turn traffic in the opposite direction.

Design and Operation

The FSM operates through **eight states**, transitioning to manage traffic flow for vehicles and pedestrians effectively:

1. **State 1**:
    - Phase 1 vehicle lights turn green, Phase 2 vehicle lights turn red.
    - Pedestrian lights synchronize with their respective phases (Phase 1 pedestrian lights red, Phase 2 pedestrian lights green).
2. **State 2**:
    - A 15-second delay is implemented using a clock divider.

3. **State 3**:
    - All vehicle lights change to yellow, and pedestrian lights remain red.
4. **State 4**:

    - A 3-second delay is introduced for the yellow light transition.
5. **State 5**:
    - Phase 1 vehicle lights turn red, and Phase 2 vehicle lights turn green.
    - Pedestrian lights switch to match their respective phases.
6. **State 6**:
    - Another 15-second delay is applied.
7. **State 7**:
    - Phase 3 is activated, turning on two LEDs (Red, Green) for left-turn traffic while Phases 1 and 2 remain red.
8. **State 8**:
    - Phase 4 is activated, functioning similarly to Phase 3, but for left-turn traffic in the opposite direction.

Hardware Configuration

- **LED Connections:**
- The system uses high-power 220V LEDs, requiring relays for safe and reliable operation.
- All LEDs share a common ground for simplicity.
- Similar LED groups (e.g., all red lights in Phase 1) are connected to dedicated relays.
- Yellow lights across all phases share a single relay.

- **Timing Management:** A clock divider generates precise delays for each phase transition, ensuring accurate timing for 15-second and 3-second intervals.

2)Ultrasonic Sensor (HC-SR04):

Overview:

The ultrasonic sensor operates by emitting high-frequency sound waves and calculating the time it takes for the waves to bounce back from an object. The VHDL design employs a Finite State Machine (FSM) to control the sensor, manage timing signals, and compute distances based on the time of flight. The system outputs the calculated distance in centimeters, formatted for integration with other components. Additionally, the design has been extended to incorporate a multi-ultrasonic system that utilizes four sensors (A1, A2, B1, and B2) working together. The system monitors multiple lanes and activates an alert if a vehicle is detected passing while the traffic light is red.

Initialization:

To operate the multi-ultrasonic sensor system and begin measuring distances:

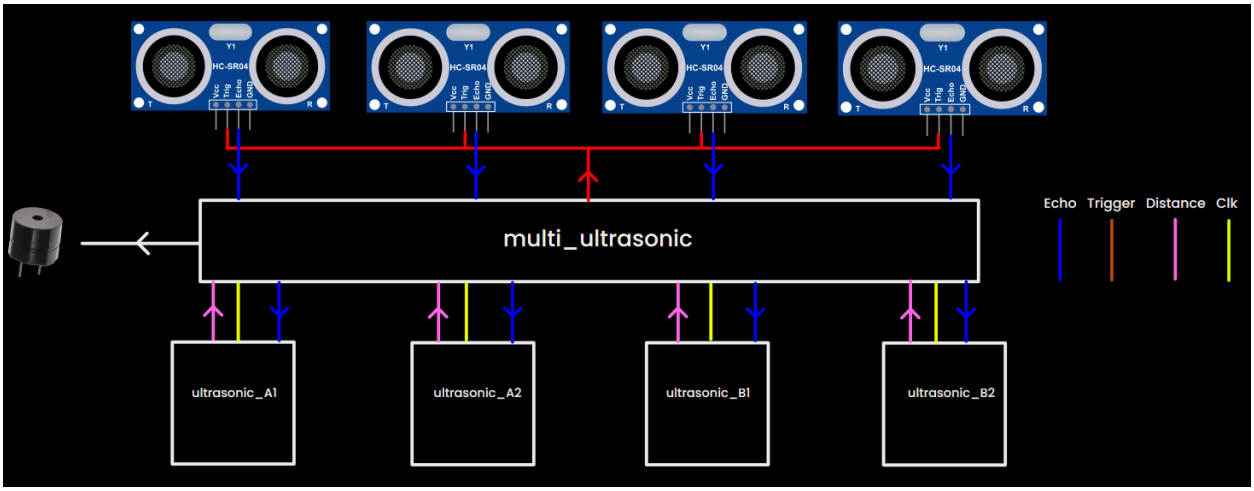
1. The system is reset using the rst signal, preparing the sensors and FSM for operation.
2. The FSM sends a 10 µs high pulse to the trigger lines of all sensors, signaling them to emit sound waves.
3. The FSM initializes all counters and variables to handle incoming echo signals from all sensors, ensuring precise measurements.

Sending and Receiving Data:

1. **Control Signals:**
    - **Trigger Signal:** A 10 µs pulse is generated on each sensor's trigger line to start measurements at same moment.
    - **Echo Signal:** Each sensor returns an echo pulse whose duration corresponds to the distance to an object.

2. **Data Transmission:**
    - Each sensor calculates the round-trip time of the sound wave, which is used to compute the distance:

Design and Operation:



1) Using Finite State Machine (FSM):

To control the multi-ultrasonic sensors:

- **zero_state:** Initializes outputs and prepares variables for a new measurement cycle across all sensors.
- **trigger_state:** Sends a 10 µs high pulse on the trigger lines to start the operation of all sensors.
- **wait_for_echo_state:** Waits for the echo signals from each sensor to begin and starts timing the pulse duration.
- **calc_distance_state:** Calculates the distance based on the echo signals' durations for all sensors.
- **output_state:** Converts the calculated distances to 24-bit values and outputs them on the respective distance ports, formatted for integration with the shared common_bus.

2) Timing Delays:

- **Trigger Pulse Duration:** A precise 10 µs pulse is generated for each sensor.
- **Echo Timing:** Counters measure the duration of each echo signal to calculate the distances.
- **Measurement Cycle Delay:** A 45 ms delay is introduced between measurement cycles to avoid interference or sensor instability.

3) Data Processing and Common Bus Interface:

- **Data Formatting:**
  - The distances (in cm) from all sensors are computed and stored as 24-bit binary values.
  - Alerts are raised if a vehicle crosses a red-light traffic lane, with the status reflected on the shared common_bus.
- **Common Bus Interface:**
  - The shared bus manages communication between sensors and system components, including traffic light signals and buzzer activation.

4) Traffic Monitoring and Alert Mechanism:

- The system continuously checks traffic lane statuses using the common_bus.
- If a red light (Lane_A_Red or Lane_B_Red) is active and a vehicle is detected within the alert distance by any sensor, the buzzer is activated for warning. This mechanism ensures that any vehicle passing through a red-light lane is detected promptly, and an alert is triggered.

3)Temperature Sensor (DHT11):

Overview:

The DHT11 sensor communicates with a microcontroller using a single-wire protocol. It transmits a 40-bit data packet containing temperature and humidity values, along with a checksum for data integrity. In the VHDL design, a Finite State Machine (FSM) controls the communication with the sensor, processing the data bit by bit and validating the checksum to ensure accuracy.

To start working with the DHT11 we needed to :

- **Initialization:** To operate the DHT11 sensor:

1. The sensor is powered on and reset.
2. The communication sequence begins with the microcontroller pulling the data line low for 18ms to initiate communication.
3. The sensor responds with an 80µs low pulse followed by an 80µs high pulse.
4. The FSM initializes all outputs and variables for a new communication cycle, readying the system to read data from the sensor.

- **Sending and Receiving Data through the :**

  1)Control Signals:

  - **dht_data_dir:** Controls the direction of data flow on the data line, toggling between input and output modes.
  - **EN (Enable):** Used to manage data reading and writing operations in coordination with the timing signals.

2)Data Transmission:

- The DHT11 sensor transmits a 40-bit data packet containing:
  - 16 bits for humidity (8 bits integer and 8 bits decimal).
  - 16 bits for temperature (8 bits integer and 8 bits decimal).
  - 8 bits for checksum.
- Data is transmitted bit by bit, with the duration of the high pulse determining if the bit is a '0' (short pulse) or a '1' (long pulse).

Design and Operation

1. **Using Finite State Machine:**
   - A state machine is designed with the following states:
     - **zero state:** Initializes outputs and prepares variables for communication.
     - **start_down_state:** Pulls the data line low to signal the start of communication.
     - **start_up_state:** Releases the data line and waits for the sensor's response.
     - **waitResponse_state:** Waits for the sensor's acknowledgment signal.
     - **readData_state:** Reads the data transmitted by the sensor and stores it in a buffer.
     - **process_state:** Extracts and processes the humidity, temperature, and checksum, calculating the checksum to validate data integrity.

        - **assign_output_state:** If the checksum is valid, the temperature and humidity data are formatted for output and displayed in readable format.

2. **Timing Delays:**
    - Delays are managed using counters to ensure proper timing of transitions between states and control signals like EN.
    - A small delay is required after sending each nibble of data to allow the sensor to process the information.
3. **Nibble-by-Nibble Transfer:**
    - The 40-bit data packet is divided into nibbles (4 bits), with the higher 4 bits sent first, followed by the lower 4 bits.
    - Each nibble is transmitted sequentially, with precise control of the dht_data_dir signal to toggle between input and output modes.
4. **Data Processing and Checksum Validation:**
    - The humidity and temperature data are extracted from the received bits.
    - The checksum is calculated and compared with the received checksum to ensure data accuracy.
    - If the checksum is invalid, an error flag is raised.
5. **Common Bus Interface:**
    - The processed data is transferred via a shared common bus for integration with other system components.
    - This allows dynamic communication of the temperature and humidity data for further use in the system

4)Liquid Crytal Display (LCD):

Overview:

The communication design for the LCD module in VHDL employs a Finite State Machine (FSM) to oversee the sequence of operations necessary for interfacing with an LCD display. This approach guarantees precise data transmission and display updates, facilitating the correct initialization, writing, and clearing of the display through a parallel communication protocol. The LCD can function in either 4-bit or 8-bit mode for sending commands and data. In 4-bit mode, only half of a byte—either the upper or lower 4 bits—is transmitted at once, thereby minimizing the number of data lines needed for interfacing with the LCD.

How the LCD Works in 4-bit Mode and How It is Operated in VHDL:

To use the LCD in 4-bit mode:

- **Initialization:**

1. The LCD is powered up and reset.

2. The initialization sequence involves sending specific commands in 8-bit mode to switch to 4-bit mode.

3. Once in 4-bit mode, commands and data are sent as two nibbles (4 bits each).

- **Sending Commands and Data:**

1)Control Signals:

RS (Register Select): Determines whether data is a command (RS=0) or character (RS=1).

RW (Read/Write): Specifies whether data is being written (RW=0) or read (RW=1).

EN (Enable): Acts as a latch; data is written when this signal transitions from HIGH to LOW.

2)4-Bit Data Transmission:

The higher 4 bits of a byte are sent first, followed by the lower 4 bits.

After sending each nibble, a small delay is required to let the LCD process the data.

Design and Operation

1)Using Finite State Machine :

- The state machine was designed with states such as INIT, WRITE, READY, and CLEAR to control the sequence of operations.
- Each state performs specific tasks like initialization, sending commands, or displaying characters.

2)Timing Delays:

Delays were implemented using counters to ensure proper timing for enabling signals (EN) and transitions between states.

3)Nibble-by-Nibble Transfer:

- Data to be displayed is stored in a buffer and divided into higher and lower nibbles.
- Each nibble is sent to the LCD sequentially with proper control signal adjustments.

4)Character Display:

- ASCII values of characters were pre-loaded into a result register.
- These characters are sent one by one to the LCD using the state machine.

5)Common Bus Interface:

The bus allows the FSM to send the processed temperature, humidity, or other relevant data to the LCD for display. As it used for sharing data, where ASCII values were transmitted to display characters dynamically.

This approach ensures efficient operation of the LCD in 4-bit mode while minimizing the use of GPIO pins and adhering to timing constraints.

5) Main Project Integration and Wiring Efficiency:

A) Main Project Integration

1. **Modular Design:**
    - Each subsystem (ultrasonic, DHT11, traffic lights, LCD) is encapsulated in dedicated modules. This modularity allows for:
        - Easier debugging and testing of individual parts.
        - Independent upgrades or changes to specific modules without affecting others.
        - Grouping the ultrasonic trigger and echo lines to reduce clutter by making for all of them a common trigger to send the signal once to avoid any delays or miscalculations of the distance .

2. **Interoperability via Common Bus:**
    - The common_bus[63:0] serves as the backbone for data exchange between modules.
    - For example:
        - The DHT11 module sends temperature data to the LCD via the bus.
        - The ultrasonic module triggers the buzzer and updates the traffic controller if a red-light violation is detected.

3. **Data Organization:**
    - The data format (e.g., temperature, traffic state, alerts) ensures efficient use of bus bandwidth, reducing delays in processing and updating system states.

4. **Integration of Safety Features:**
    - The ultrasonic sensors directly interface with the traffic controller and buzzer to enforce safety (red-light violation detection).
    - The camera (in the first diagram) can be added to this setup seamlessly by integrating its alert mechanism with the ultrasonic_inst module.

5. **LCD as a Multi-Purpose Component:**
    - The integration of an LCD for both displaying temperature and acting as an advertisement poster adds value to the project, making it both functional and commercially viable.

6. **Synchronization Across Modules:**
    - The use of a reset signal (reset) ensures a synchronized startup, reducing risks of miscommunication between modules during initialization.

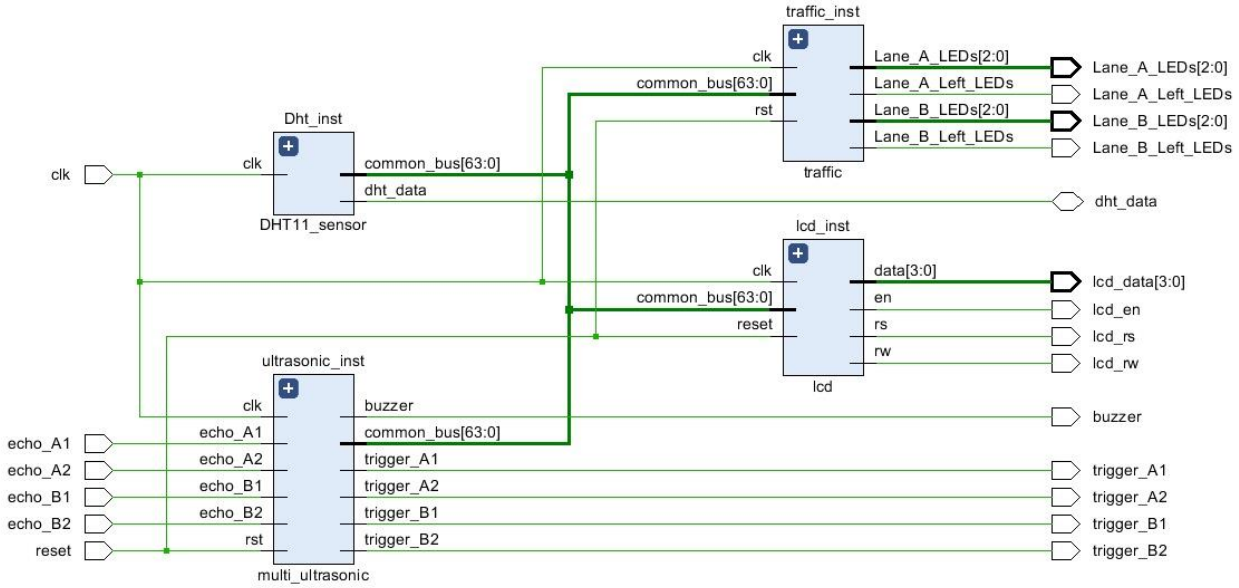
B) Wiring Efficiency:

1. **Common Bus Architecture:**

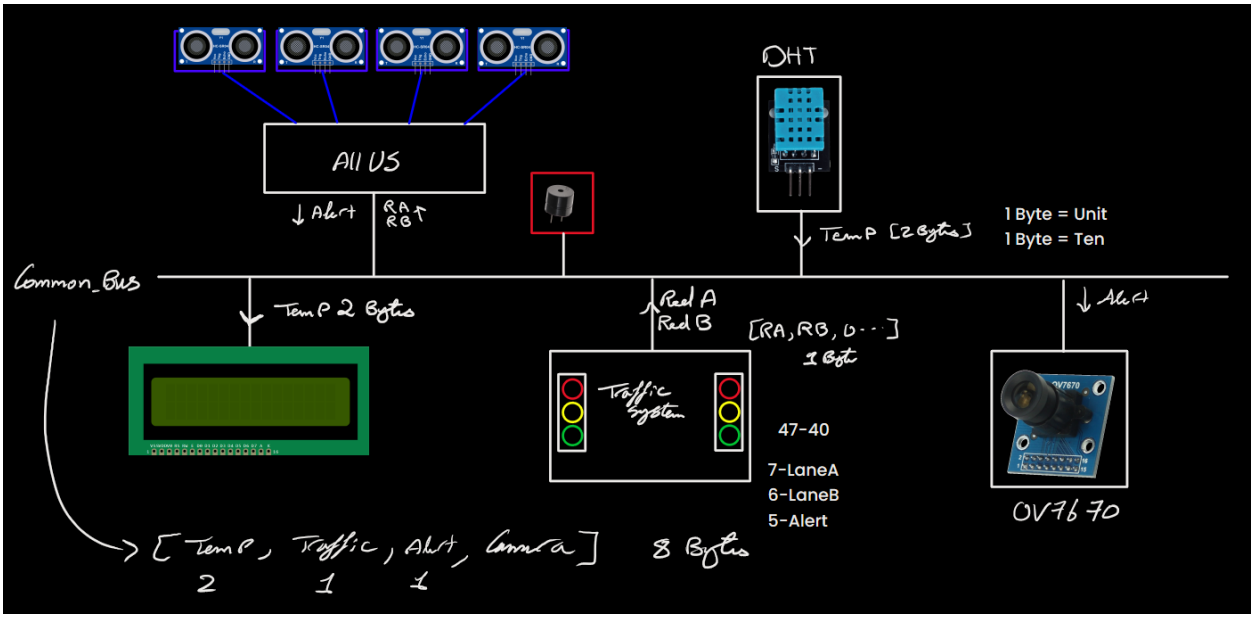
- Both diagrams utilize a **common bus system** (common_bus[63:0]), which consolidates communication lines between various modules.
- This significantly reduces wiring complexity, as the sensors, actuators, and display units share the same communication interface.
- The bus architecture ensures scalability, allowing new modules to be added with minimal changes to the wiring.

2. **Module-Specific Connections:**
   - Ultrasonic sensors have individual trigger and echo connections for precise detection. These are directly wired to the ultrasonic_inst module, which processes signals locally and sends alerts via the common bus.
   - The LCD module (lcd_inst) uses a clean interface with data[3:0], en, rs, and rw signals, ensuring simple and efficient control wiring for the display.
   - The traffic light controller (traffic_inst) has separate outputs for each lane and direction (Lane_A_LEDs, Lane_B_LEDs), which helps in maintaining a clear structure for controlling multiple lights.
3. **Reduced Redundancy:**
   - By centralizing the logic in modules like DHT11_sensor, ultrasonic_inst, and traffic_inst, the system avoids repetitive wiring of control signals, as each module handles its function independently.
4. **Synchronization:**

- Using a shared clk (clock) signal ensures all modules operate in sync, further simplifying wiring by avoiding additional timing circuits.

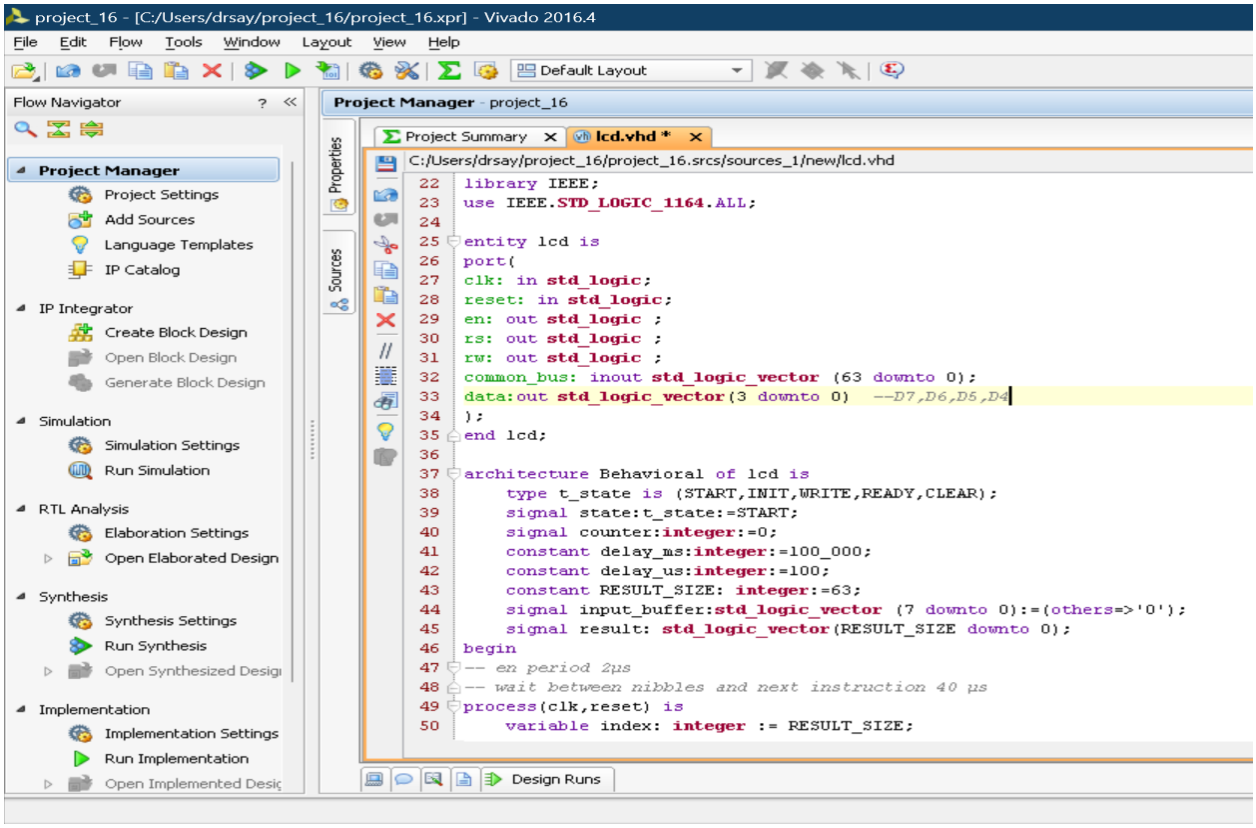


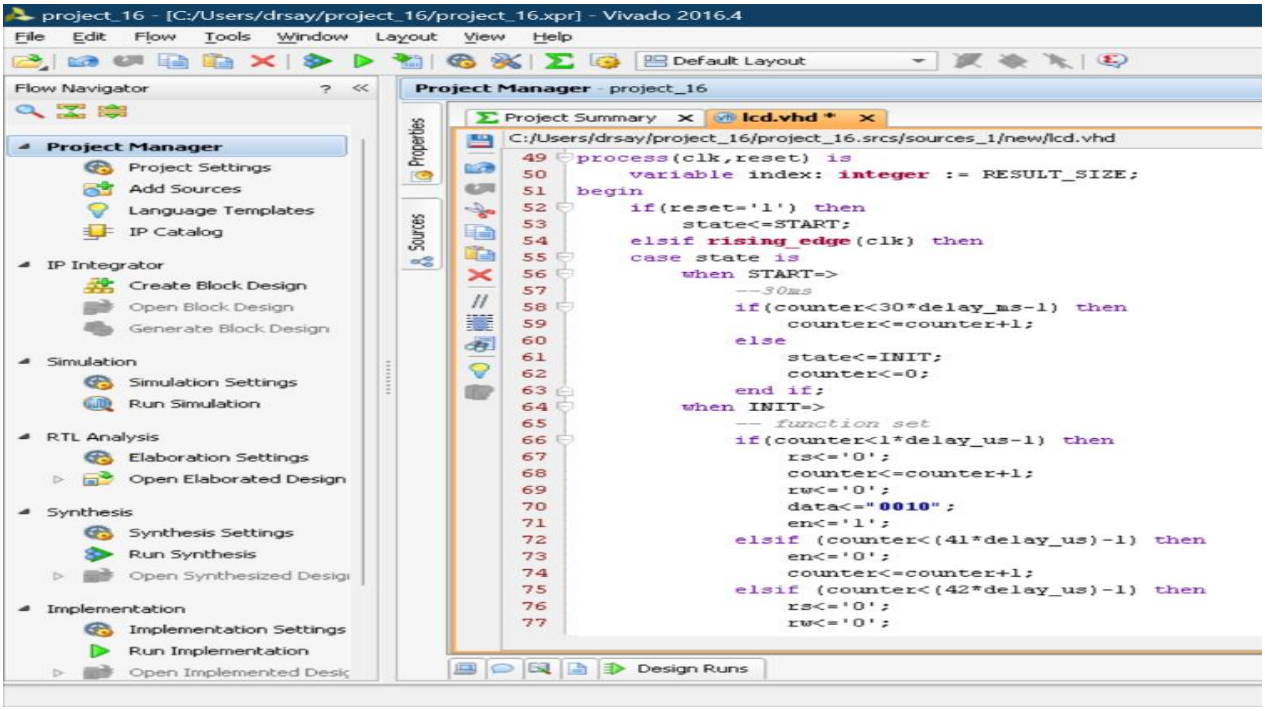
C)A Schematic shows the Smart Traffic System including Integration of Sensors and Display Interfaces:



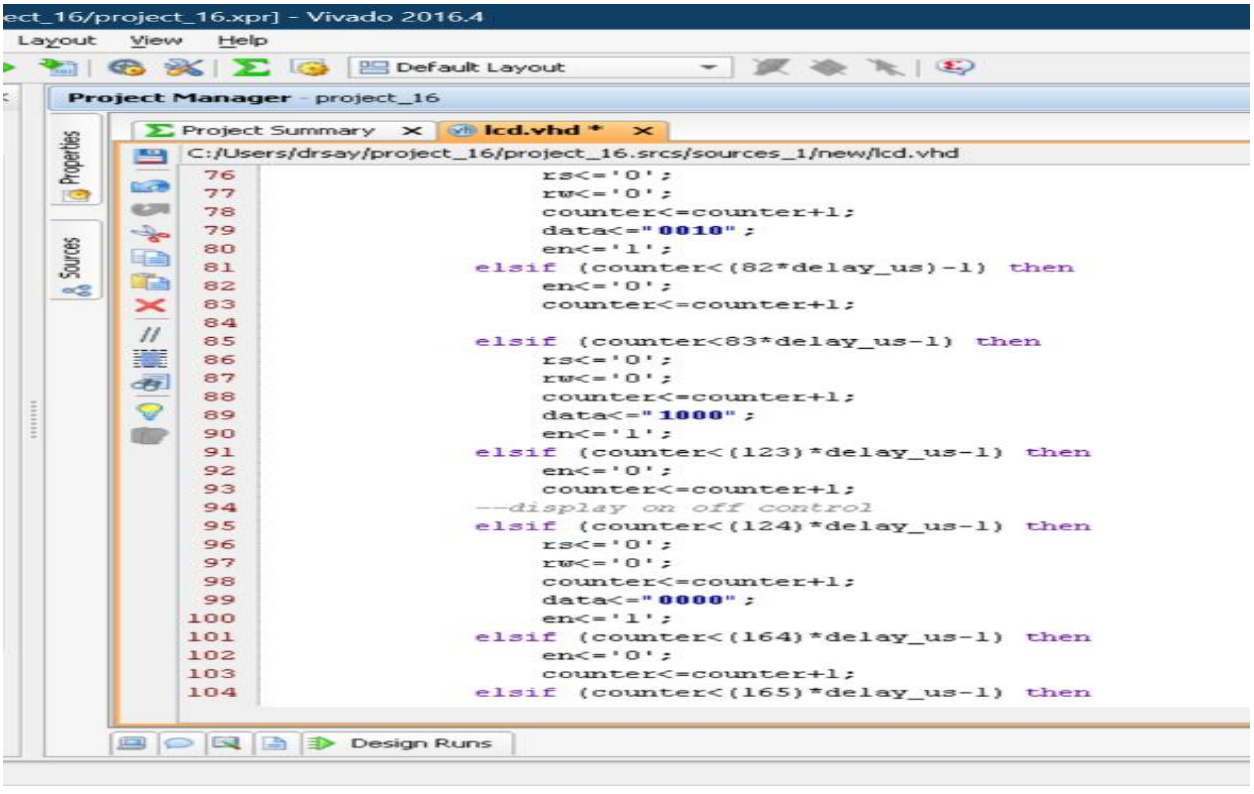
Implementation and Simulation:

For LCD:

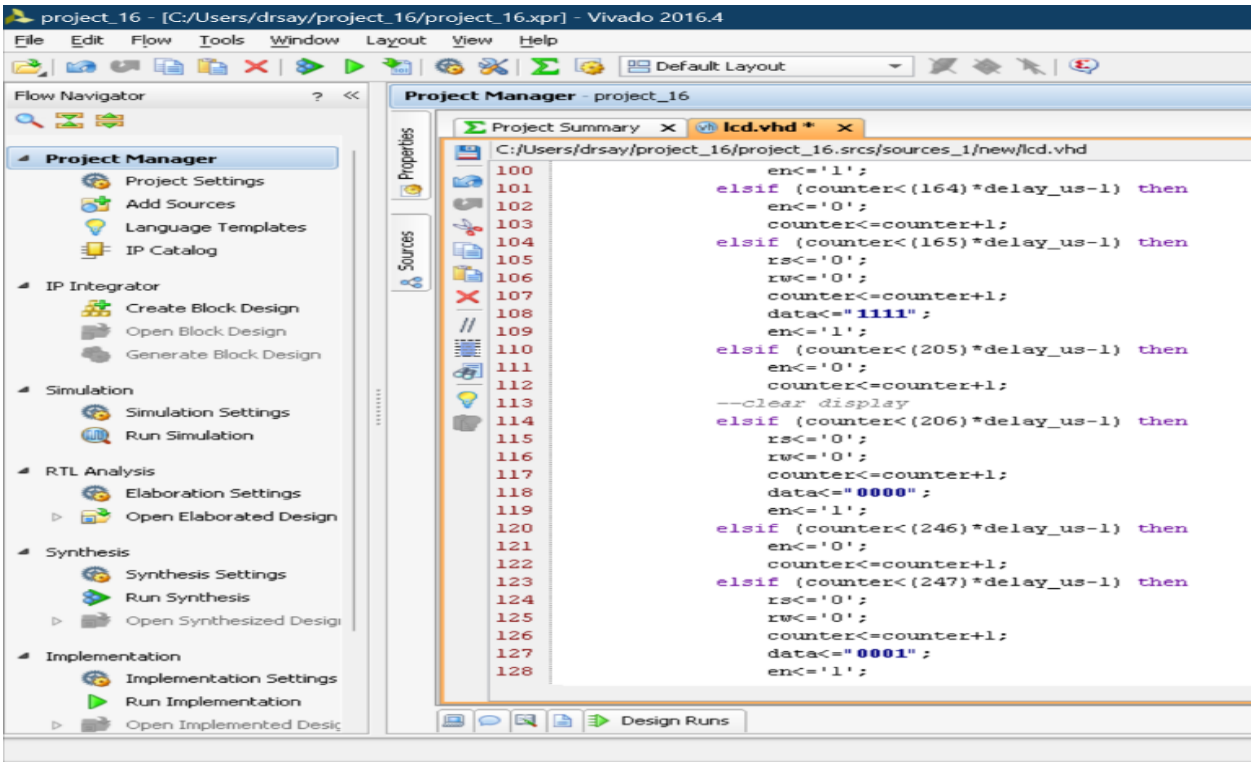




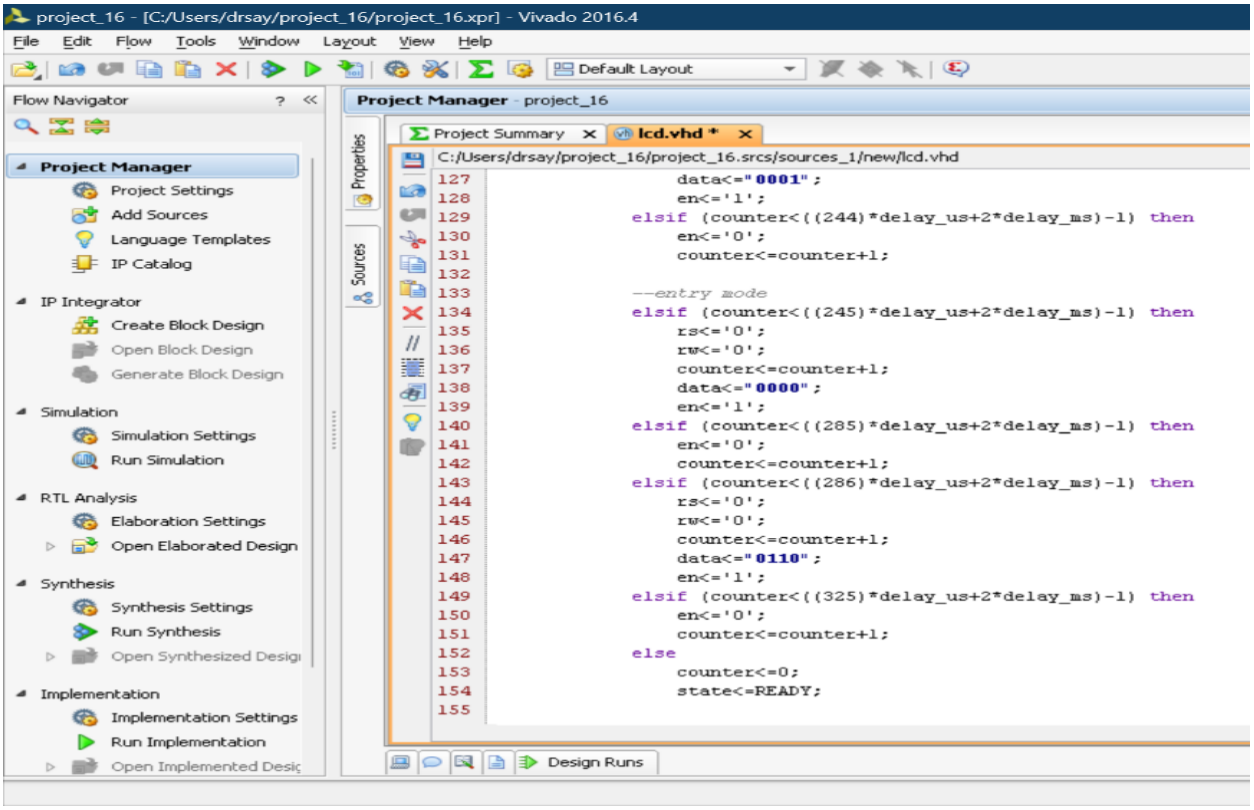

```
49  process(clk,reset) is
50      variable index: integer := RESULT_SIZE;
51  begin
52      if(reset='1') then
53          state<=START;
54      elsif rising_edge(clk) then
55      case state is
56          when START=>
57              --30ms
58              if(counter<30*delay_ms-1) then
59                  counter<=counter+1;
60              else
61                  state<=INIT;
62                  counter<=0;
63              end if;
64          when INIT=>
65              -- function set
66              if(counter<1*delay_us-1) then
67                  rs<='0';
68                  counter<=counter+1;
69                  rw<='0';
70                  data<="0010";
71                  en<='1';
72              elsif (counter<(41*delay_us)-1) then
73                  en<='0';
74                  counter<=counter+1;
75              elsif (counter<(42*delay_us)-1) then
76                  rs<='0';
77                  rw<='0';
```



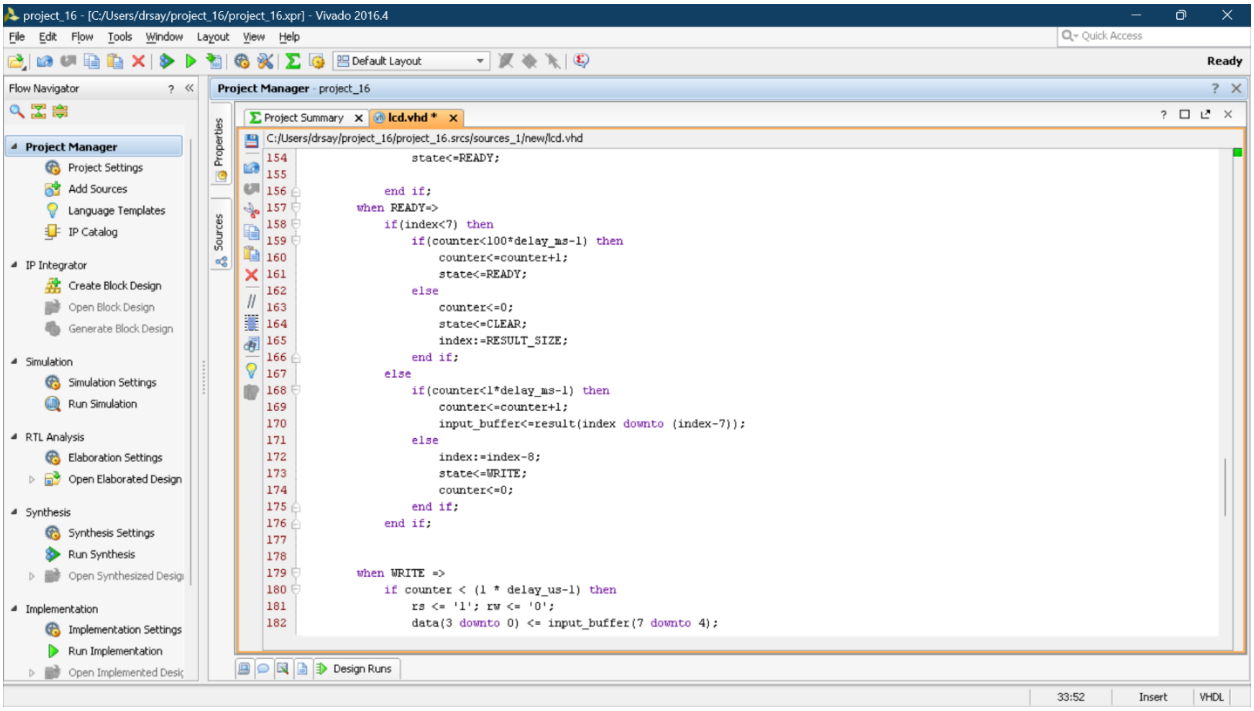

```
76                      rs<='0';
77                      rw<='0';
78                      counter<=counter+1;
79                      data<="0010";
80                      en<='1';
81                  elsif (counter<(82*delay_us)-1) then
82                      en<='0';
83                      counter<=counter+1;
84
85                  elsif (counter<83*delay_us-1) then
86                      rs<='0';
87                      rw<='0';
88                      counter<=counter+1;
89                      data<="1000";
90                      en<='1';
91                  elsif (counter<(123)*delay_us-1) then
92                      en<='0';
93                      counter<=counter+1;
94                  --display on off control
95                  elsif (counter<(124)*delay_us-1) then
96                      rs<='0';
97                      rw<='0';
98                      counter<=counter+1;
99                      data<="0000";
100                     en<='1';
101                 elsif (counter<(164)*delay_us-1) then
102                     en<='0';
103                     counter<=counter+1;
104                 elsif (counter<(165)*delay_us-1) then
```



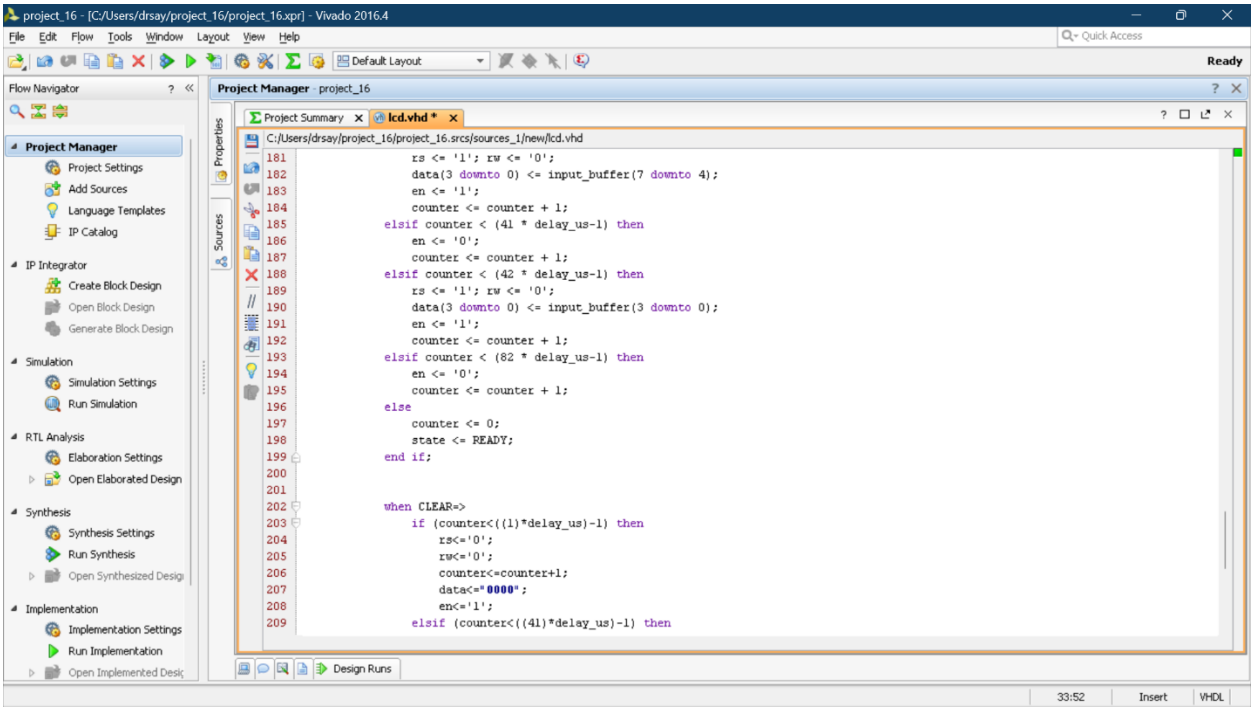

```
100                     en<='1';
101                 elsif (counter<(164)*delay_us-1) then
102                     en<='0';
103                     counter<=counter+1;
104                 elsif (counter<(165)*delay_us-1) then
105                     rs<='0';
106                     rw<='0';
107                     counter<=counter+1;
108                     data<="1111";
109                     en<='1';
110                 elsif (counter<(205)*delay_us-1) then
111                     en<='0';
112                     counter<=counter+1;
113                 --clear display
114                 elsif (counter<(206)*delay_us-1) then
115                     rs<='0';
116                     rw<='0';
117                     counter<=counter+1;
118                     data<="0000";
119                     en<='1';
120                 elsif (counter<(246)*delay_us-1) then
121                     en<='0';
122                     counter<=counter+1;
123                 elsif (counter<(247)*delay_us-1) then
124                     rs<='0';
125                     rw<='0';
126                     counter<=counter+1;
127                     data<="0001";
128                     en<='1';
```



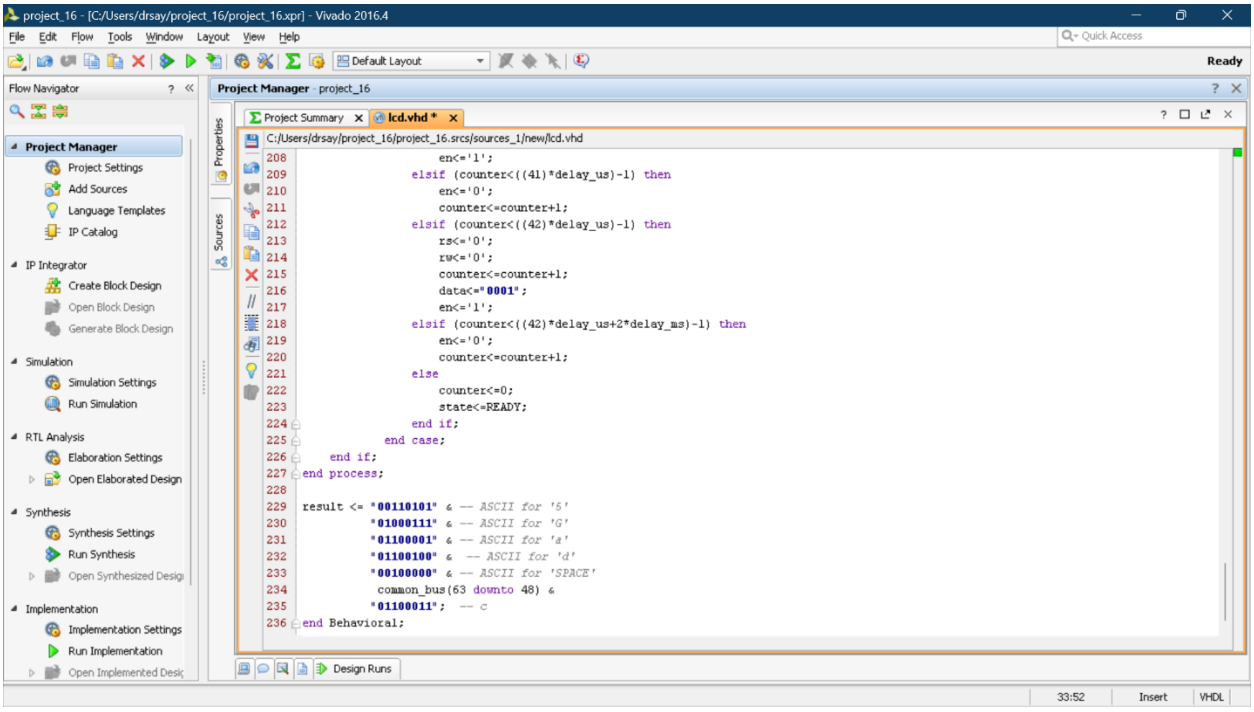

```
127                     data<="0001";
128                     en<='1';
129                 elsif (counter<((244)*delay_us+2*delay_ms)-1) then
130                     en<='0';
131                     counter<=counter+1;
132
133                 --entry mode
134                 elsif (counter<((245)*delay_us+2*delay_ms)-1) then
135                     rs<='0';
136                     rw<='0';
137                     counter<=counter+1;
138                     data<="0000";
139                     en<='1';
140                 elsif (counter<((285)*delay_us+2*delay_ms)-1) then
141                     en<='0';
142                     counter<=counter+1;
143                 elsif (counter<((286)*delay_us+2*delay_ms)-1) then
144                     rs<='0';
145                     rw<='0';
146                     counter<=counter+1;
147                     data<="0110";
148                     en<='1';
149                 elsif (counter<((325)*delay_us+2*delay_ms)-1) then
150                     en<='0';
151                     counter<=counter+1;
152                 else
153                     counter<=0;
154                     state<=READY;
155
```



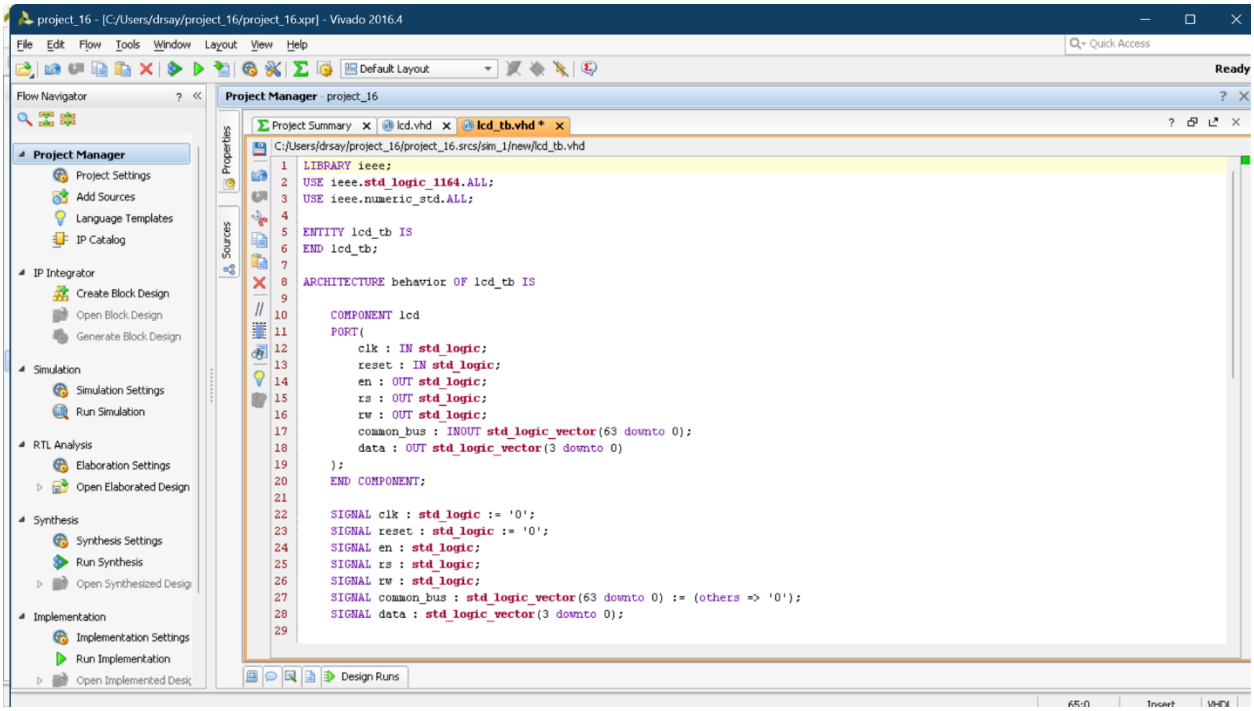

```
154                 state<=READY;
155
156             end if;
157         when READY=>
158             if(index<7) then
159                 if(counter<100*delay_ms-1) then
160                     counter<=counter+1;
161                     state<=READY;
162                 else
163                     counter<=0;
164                     state<=CLEAR;
165                     index:=RESULT_SIZE;
166                 end if;
167             else
168                 if(counter<1*delay_ms-1) then
169                     counter<=counter+1;
170                     input_buffer<=result(index downto (index-7));
171                 else
172                     index:=index-8;
173                     state<=WRITE;
174                     counter<=0;
175                 end if;
176             end if;
177
178
179         when WRITE =>
180             if counter < (1 * delay_us-1) then
181                 rs <= '1'; rw <= '0';
182                 data(3 downto 0) <= input_buffer(7 downto 4);
```

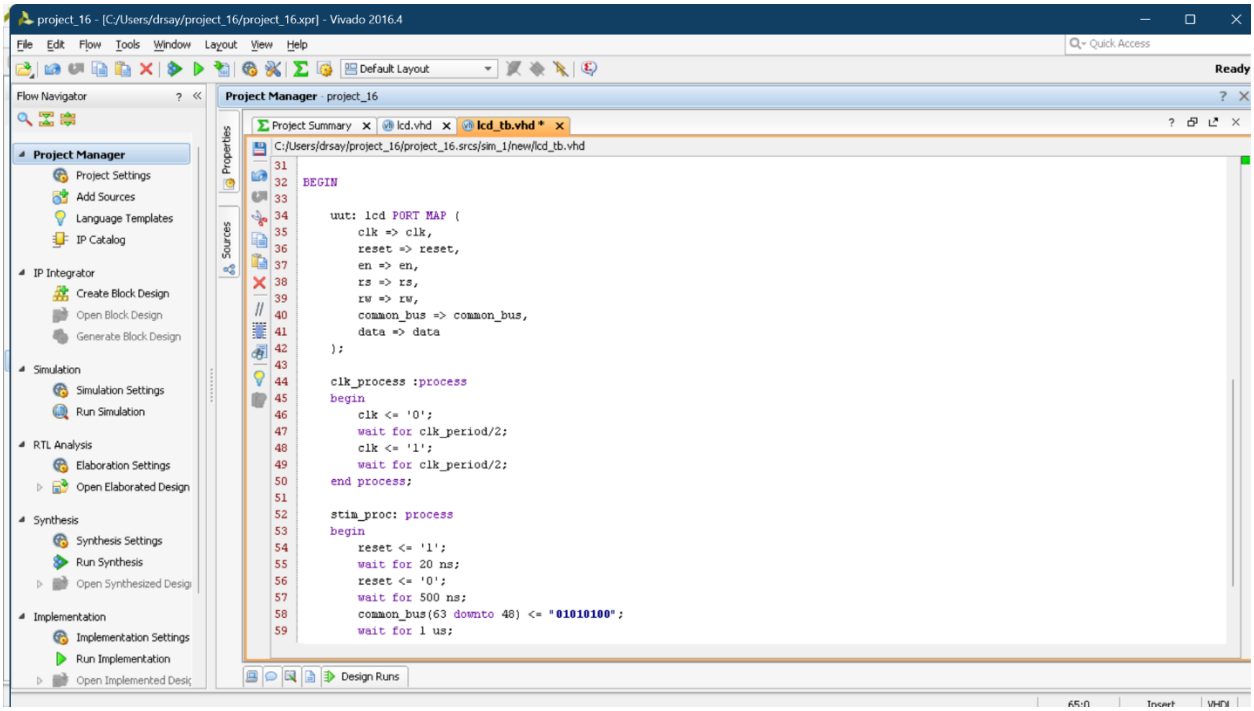


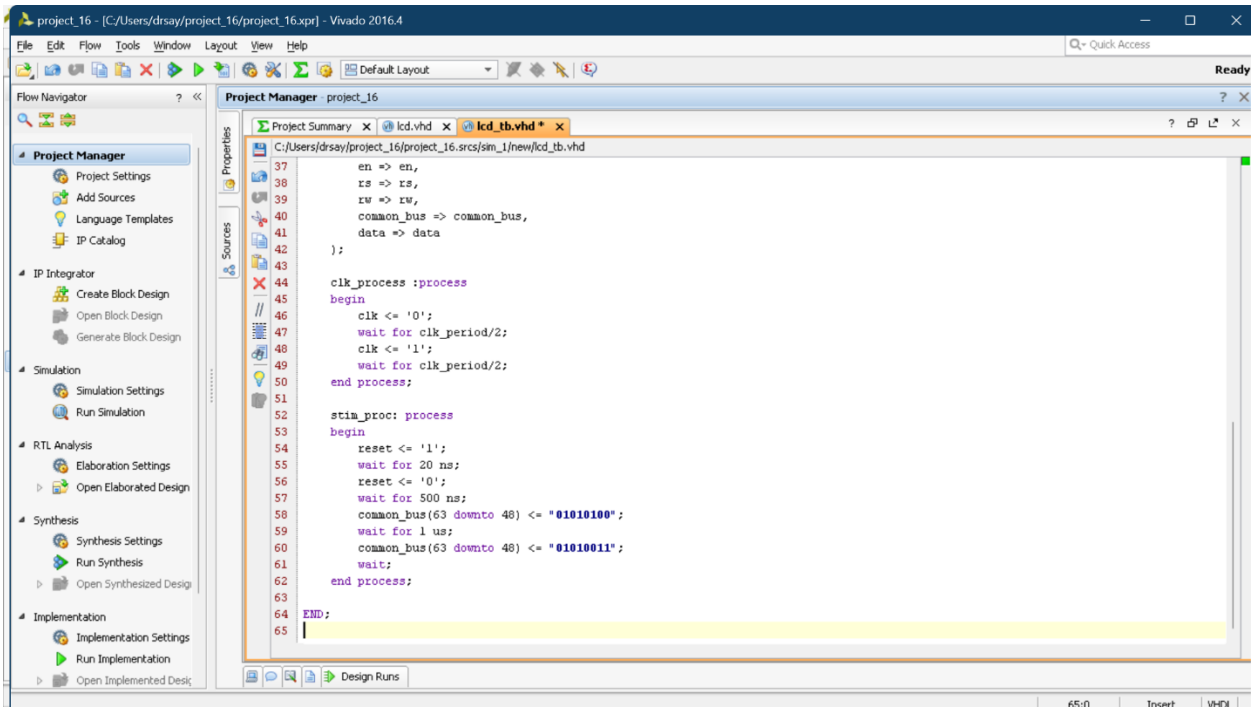

```
181                 rs <= '1'; rw <= '0';
182                 data(3 downto 0) <= input_buffer(7 downto 4);
183                 en <= '1';
184                 counter <= counter + 1;
185             elsif counter < (41 * delay_us-1) then
186                 en <= '0';
187                 counter <= counter + 1;
188             elsif counter < (42 * delay_us-1) then
189                 rs <= '1'; rw <= '0';
190                 data(3 downto 0) <= input_buffer(3 downto 0);
191                 en <= '1';
192                 counter <= counter + 1;
193             elsif counter < (82 * delay_us-1) then
194                 en <= '0';
195                 counter <= counter + 1;
196             else
197                 counter <= 0;
198                 state <= READY;
199             end if;
200
201
202         when CLEAR=>
203             if (counter<((1)*delay_us)-1) then
204                 rs<='0';
205                 rw<='0';
206                 counter<=counter+1;
207                 data<="0000";
208                 en<='1';
209             elsif (counter<((41)*delay_us)-1) then
```




```
208                     en<='1';
209                 elsif (counter<((41)*delay_us)-1) then
210                     en<='0';
211                     counter<=counter+1;
212                 elsif (counter<((42)*delay_us)-1) then
213                     rs<='0';
214                     rw<='0';
215                     counter<=counter+1;
216                     data<="0001";
217                     en<='1';
218                 elsif (counter<((42)*delay_us+2*delay_ms)-1) then
219                     en<='0';
220                     counter<=counter+1;
221                 else
222                     counter<=0;
223                     state<=READY;
224                 end if;
225             end case;
226     end if;
227 end process;
228
229 result <= "00110101" & -- ASCII for '5'
230           "01000111" & -- ASCII for 'G'
231           "01100001" & -- ASCII for 'a'
232           "01100100" &  -- ASCII for 'd'
233           "00100000" & -- ASCII for 'SPACE'
234            common_bus(63 downto 48) &
235           "01100011";  -- c
236 end Behavioral;
```

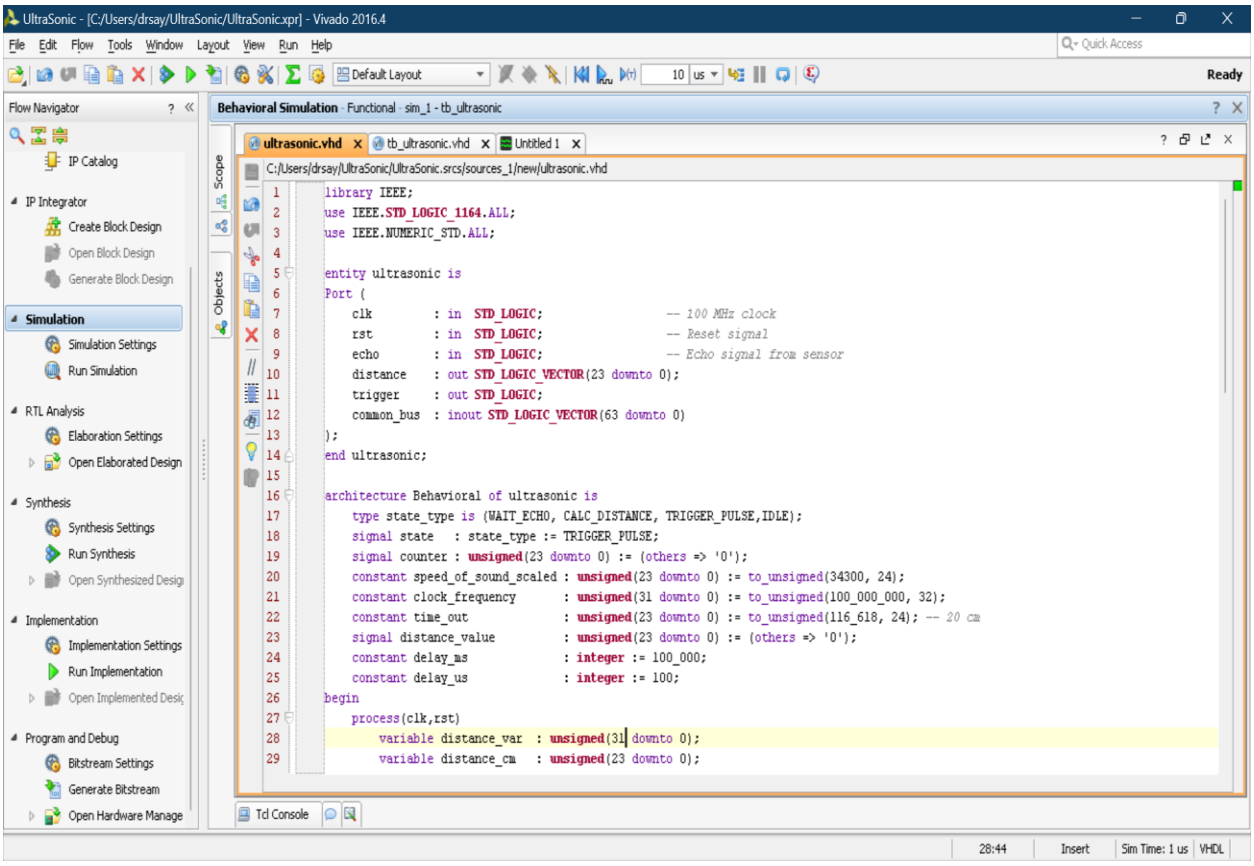

project_16 - [C:/Users/drsay/project_16/project_16.xpr] - Vivado 2016.4
File Edit Flow Tools Window Layout View Help
Quick Access
Default Layout
Ready
Flow Navigator
Project Manager
Project Settings
Add Sources
Language Templates
IP Catalog
IP Integrator
Create Block Design
Open Block Design
Generate Block Design
Simulation
Simulation Settings
Run Simulation
RTL Analysis
Elaboration Settings
Open Elaborated Design
Synthesis
Synthesis Settings
Run Synthesis
Open Synthesized Design
Implementation
Implementation Settings
Run Implementation
Open Implemented Design
Project Manager - project_16
Project Summary
lcd.vhd
lcd_tb.vhd *
C:/Users/drsay/project_16/project_16.srcs/sim_1/new/lcd_tb.vhd
Properties
Sources
Design Runs
1 LIBRARY ieee;
2 USE ieee.std_logic_1164.ALL;
3 USE ieee.numeric_std.ALL;
4
5 ENTITY lcd_tb IS
6 END lcd_tb;
7
8 ARCHITECTURE behavior OF lcd_tb IS
9
10 COMPONENT lcd
11 PORT(
12 clk : IN std_logic;
13 reset : IN std_logic;
14 en : OUT std_logic;
15 rs : OUT std_logic;
16 rw : OUT std_logic;
17 common_bus : INOUT std_logic_vector(63 downto 0);
18 data : OUT std_logic_vector(3 downto 0)
19);
20 END COMPONENT;
21
22 SIGNAL clk : std_logic := '0';
23 SIGNAL reset : std_logic := '0';
24 SIGNAL en : std_logic;
25 SIGNAL rs : std_logic;
26 SIGNAL rw : std_logic;
27 SIGNAL common_bus : std_logic_vector(63 downto 0) := (others => '0');
28 SIGNAL data : std_logic_vector(3 downto 0);
29


project_16 - [C:/Users/drsay/project_16/project_16.xpr] - Vivado 2016.4
File Edit Flow Tools Window Layout View Help
Quick Access
Default Layout
Ready
Flow Navigator
Project Manager
Project Settings
Add Sources
Language Templates
IP Catalog
IP Integrator
Create Block Design
Open Block Design
Generate Block Design
Simulation
Simulation Settings
Run Simulation
RTL Analysis
Elaboration Settings
Open Elaborated Design
Synthesis
Synthesis Settings
Run Synthesis
Open Synthesized Design
Implementation
Implementation Settings
Run Implementation
Open Implemented Design
Project Manager - project_16
Project Summary
lcd.vhd
lcd_tb.vhd *
C:/Users/drsay/project_16/project_16.srcs/sim_1/new/lcd_tb.vhd
Properties
Sources
Design Runs
31
32 BEGIN
33
34 uut: lcd PORT MAP (
35 clk => clk,
36 reset => reset,
37 en => en,
38 rs => rs,
39 rw => rw,
40 common_bus => common_bus,
41 data => data
42);
43
44 clk_process :process
45 begin
46 clk <= '0';
47 wait for clk_period/2;
48 clk <= '1';
49 wait for clk_period/2;
50 end process;
51
52 stim_proc: process
53 begin
54 reset <= '1';
55 wait for 20 ns;
56 reset <= '0';
57 wait for 500 ns;
58 common_bus(63 downto 48) <= "01010100";
59 wait for 1 us;

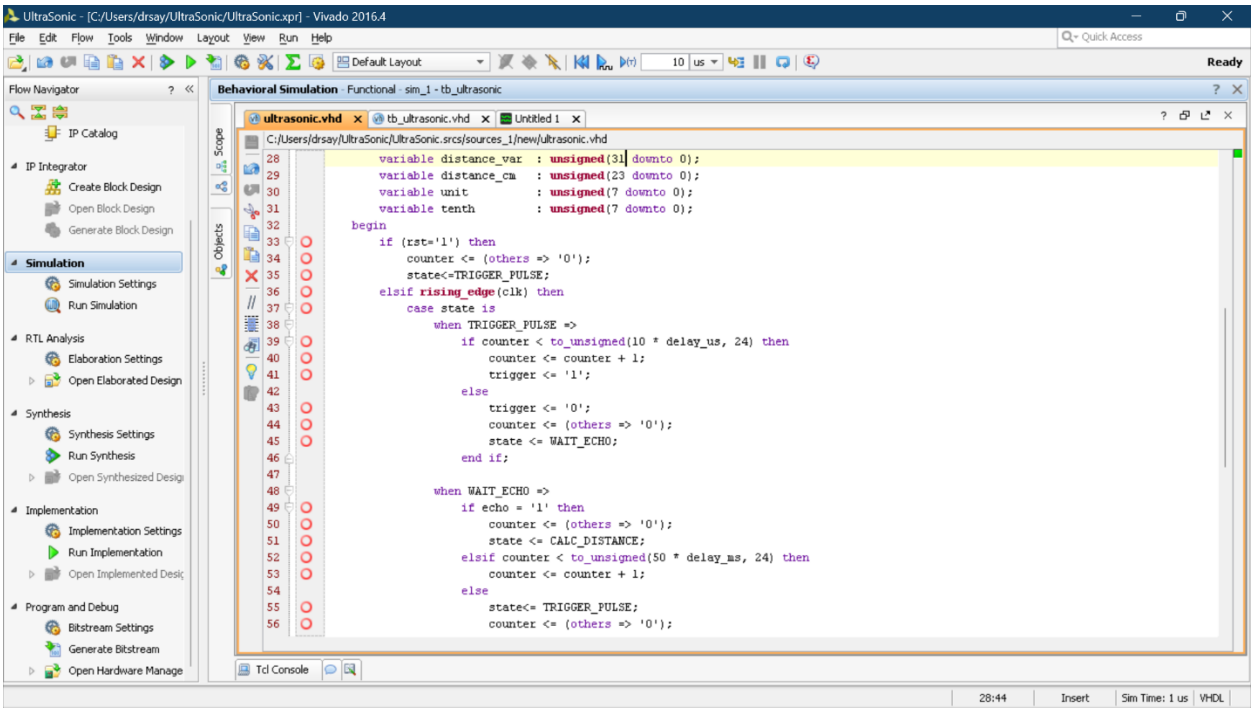



```
        en => en,
        rs => rs,
        rw => rw,
        common_bus => common_bus,
        data => data
    );

    clk_process :process
    begin
        clk <= '0';
        wait for clk_period/2;
        clk <= '1';
        wait for clk_period/2;
    end process;

    stim_proc: process
    begin
        reset <= '1';
        wait for 20 ns;
        reset <= '0';
        wait for 500 ns;
        common_bus(63 downto 48) <= "0101010";
        wait for 1 us;
        common_bus(63 downto 48) <= "0101011";
        wait;
    end process;

END;
```

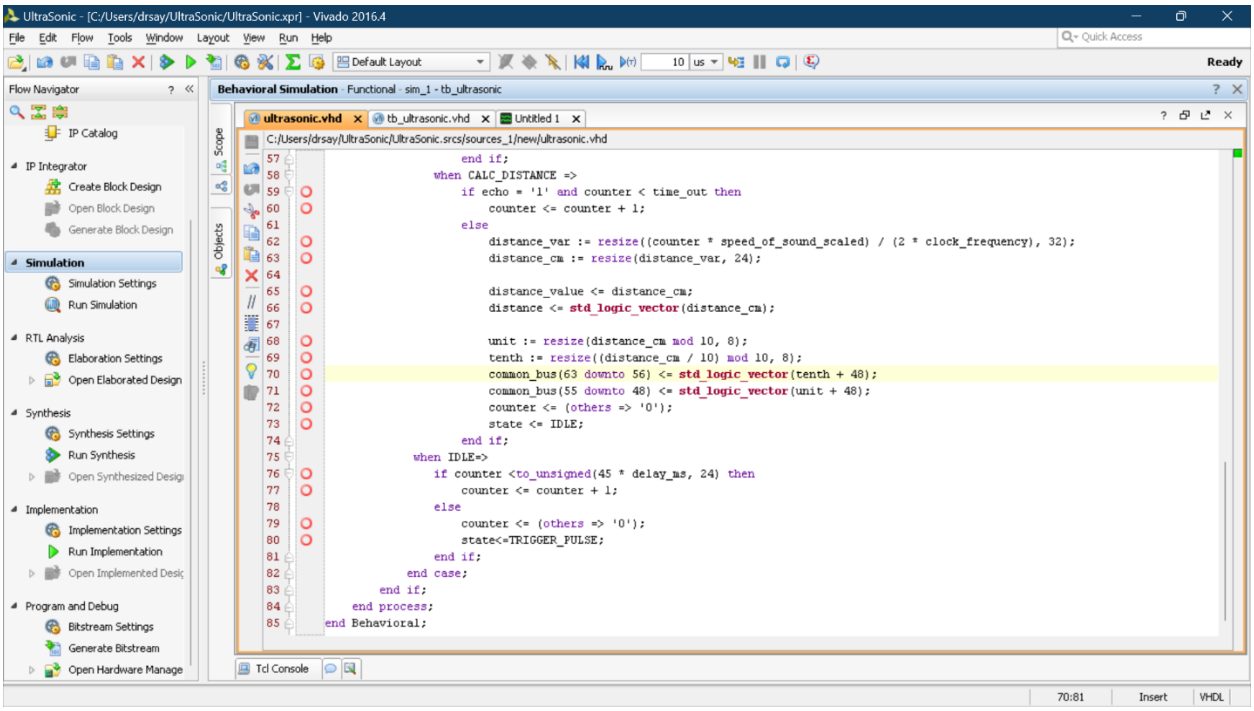
For Ultrasonic Sensor:

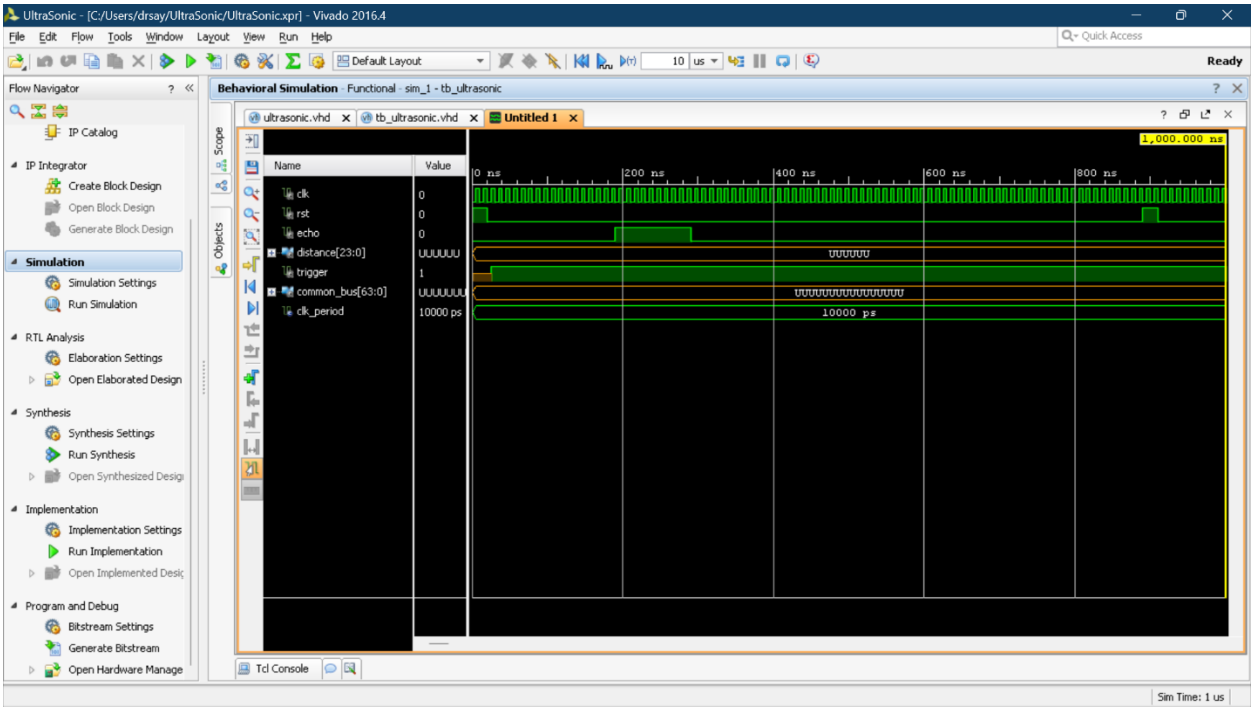


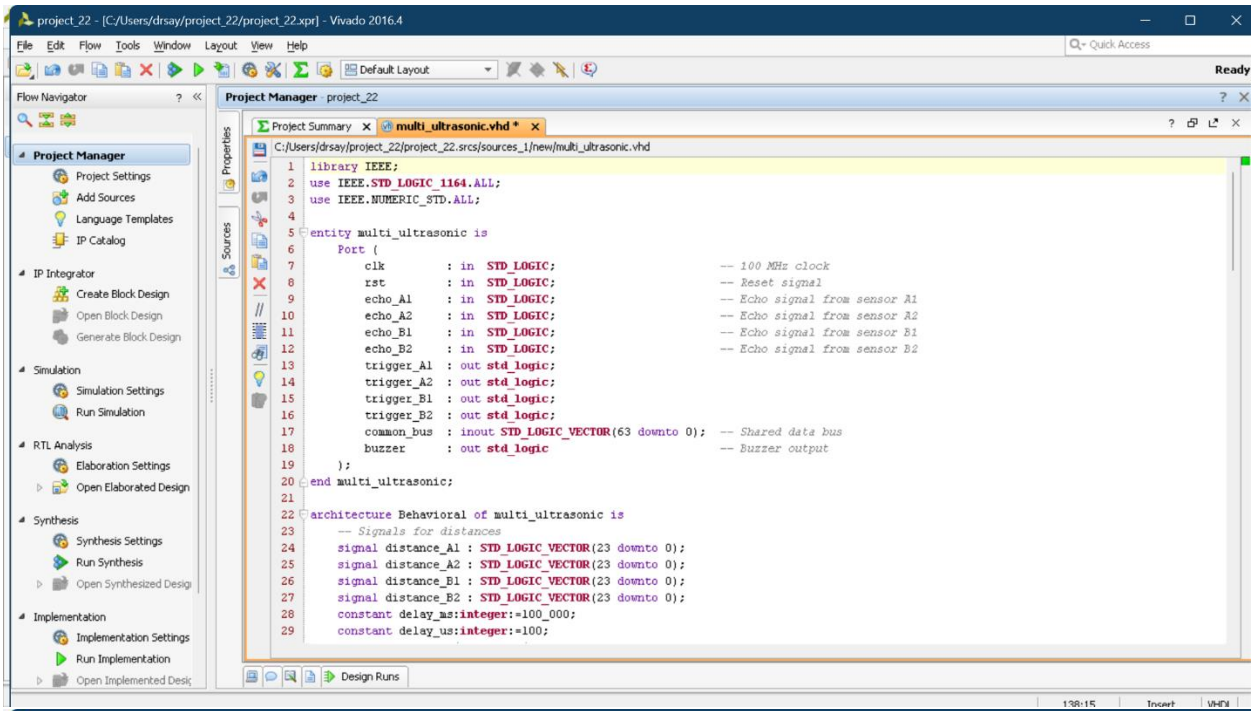

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ultrasonic is
Port (
    clk         : in  STD_LOGIC;                -- 100 MHz clock
    rst         : in  STD_LOGIC;                -- Reset signal
    echo        : in  STD_LOGIC;                -- Echo signal from sensor
    distance    : out STD_LOGIC_VECTOR(23 downto 0);
    trigger     : out STD_LOGIC;
    common_bus  : inout STD_LOGIC_VECTOR(63 downto 0)
);
end ultrasonic;

architecture Behavioral of ultrasonic is
    type state_type is (WAIT_ECHO, CALC_DISTANCE, TRIGGER_PULSE,IDLE);
    signal state   : state_type := TRIGGER_PULSE;
    signal counter : unsigned(23 downto 0) := (others => '0');
    constant speed_of_sound_scaled : unsigned(23 downto 0) := to_unsigned(34300, 24);
    constant clock_frequency       : unsigned(31 downto 0) := to_unsigned(100_000_000, 32);
    constant time_out              : unsigned(23 downto 0) := to_unsigned(116_618, 24); -- 20 cm
    signal distance_value          : unsigned(23 downto 0) := (others => '0');
    constant delay_ms              : integer := 100_000;
    constant delay_us              : integer := 100;
begin
    process(clk,rst)
        variable distance_var  : unsigned(31 downto 0);
        variable distance_cm   : unsigned(23 downto 0);
```

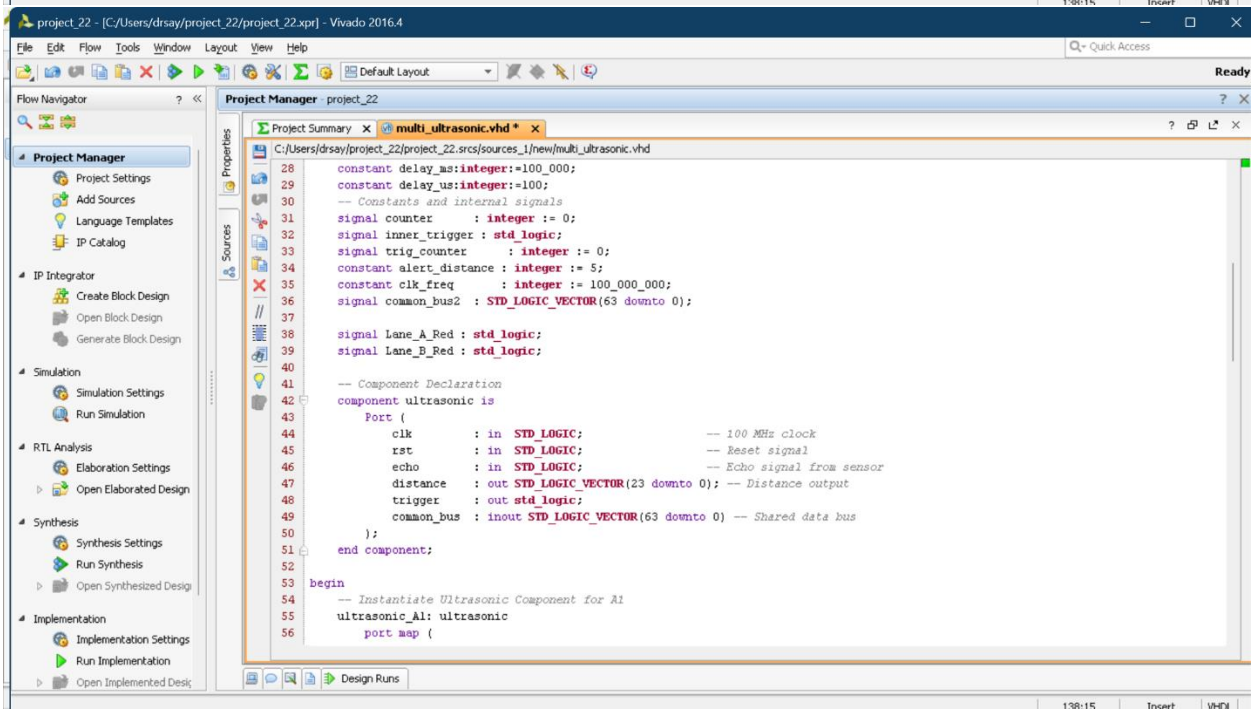


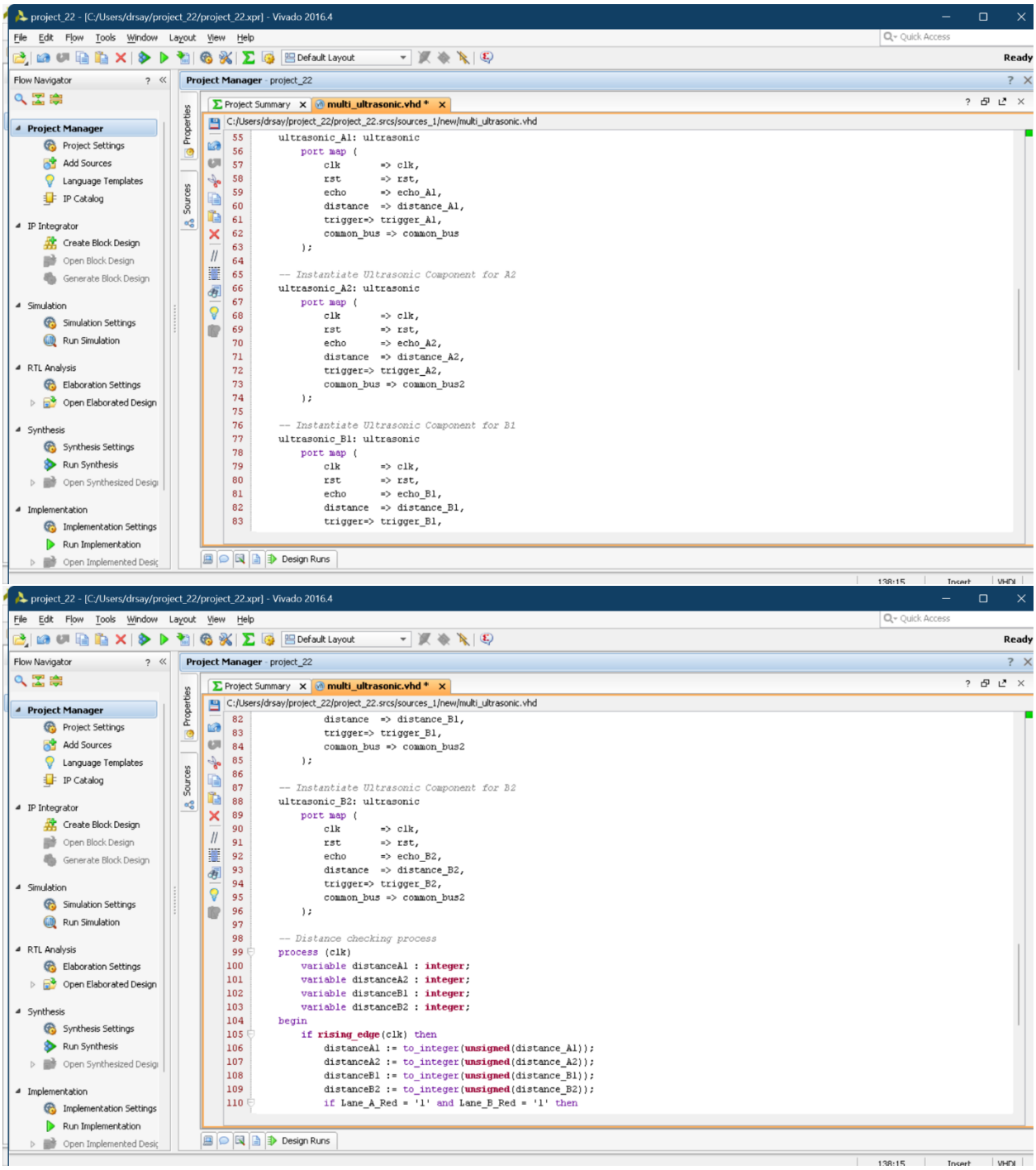

```
        variable distance_var  : unsigned(31 downto 0);
        variable distance_cm   : unsigned(23 downto 0);
        variable unit          : unsigned(7 downto 0);
        variable tenth         : unsigned(7 downto 0);
    begin
        if (rst='1') then
            counter <= (others => '0');
            state<=TRIGGER_PULSE;
        elsif rising_edge(clk) then
            case state is
                when TRIGGER_PULSE =>
                    if counter < to_unsigned(10 * delay_us, 24) then
                        counter <= counter + 1;
                        trigger <= '1';
                    else
                        trigger <= '0';
                        counter <= (others => '0');
                        state <= WAIT_ECHO;
                    end if;

                when WAIT_ECHO =>
                    if echo = '1' then
                        counter <= (others => '0');
                        state <= CALC_DISTANCE;
                    elsif counter < to_unsigned(50 * delay_ms, 24) then
                        counter <= counter + 1;
                    else
                        state<= TRIGGER_PULSE;
                        counter <= (others => '0');
```

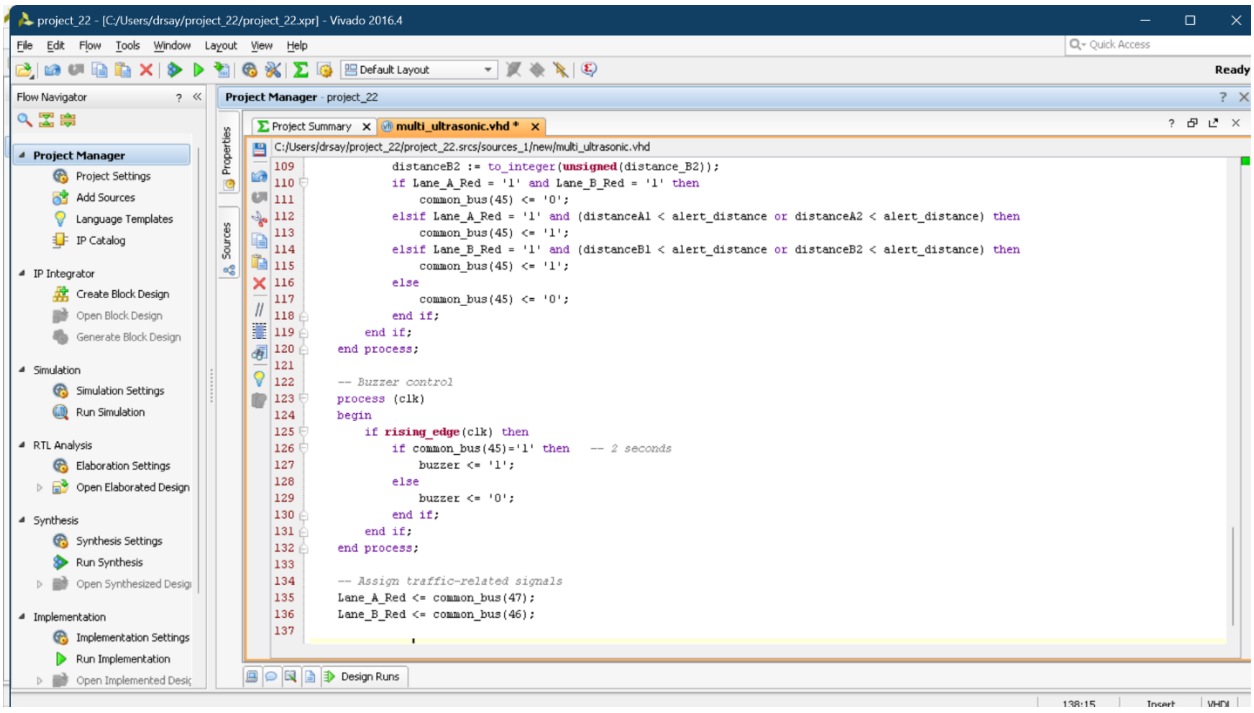

UltraSonic - [C:/Users/drsay/UltraSonic/UltraSonic.xpr] - Vivado 2016.4
File Edit Flow Tools Window Layout View Run Help
Quick Access
Default Layout
Ready
Flow Navigator
IP Catalog
IP Integrator
Create Block Design
Open Block Design
Generate Block Design
Simulation
Simulation Settings
Run Simulation
RTL Analysis
Elaboration Settings
Open Elaborated Design
Synthesis
Synthesis Settings
Run Synthesis
Open Synthesized Design
Implementation
Implementation Settings
Run Implementation
Open Implemented Design
Program and Debug
Bitstream Settings
Generate Bitstream
Open Hardware Manager
Behavioral Simulation - Functional - sim_1 - tb_ultrasonic
ultrasonic.vhd
tb_ultrasonic.vhd
Untitled 1
C:/Users/drsay/UltraSonic/UltraSonic.srcs/sources_1/new/ultrasonic.vhd
end if;
when CALC_DISTANCE =>
if echo = '1' and counter < time_out then
counter <= counter + 1;
else
distance_var := resize((counter * speed_of_sound_scaled) / (2 * clock_frequency), 32);
distance_cm := resize(distance_var, 24);
distance_value <= distance_cm;
distance <= std_logic_vector(distance_cm);
unit := resize(distance_cm mod 10, 8);
tenth := resize((distance_cm / 10) mod 10, 8);
common_bus(63 downto 56) <= std_logic_vector(tenth + 48);
common_bus(55 downto 48) <= std_logic_vector(unit + 48);
counter <= (others => '0');
state <= IDLE;
end if;
when IDLE=>
if counter <to_unsigned(45 * delay_ms, 24) then
counter <= counter + 1;
else
counter <= (others => '0');
state<=TRIGGER_PULSE;
end if;
end case;
end if;
end process;
end Behavioral;
Tcl Console
70:81
Insert
VHDL


UltraSonic - [C:/Users/drsay/UltraSonic/UltraSonic.xpr] - Vivado 2016.4
File Edit Flow Tools Window Layout View Run Help
Quick Access
Default Layout
Ready
Flow Navigator
IP Integrator
Simulation
RTL Analysis
Synthesis
Implementation
Program and Debug
Behavioral Simulation - Functional - sim_1 - tb_ultrasonic
ultrasonic.vhd
tb_ultrasonic.vhd
Untitled 1
Name
Value
clk
rst
echo
distance[23:0]
trigger
common_bus[63:0]
clk_period
10000 ps
1,000.000 ns
0 ns
200 ns
400 ns
600 ns
800 ns
Tcl Console
Sim Time: 1 us

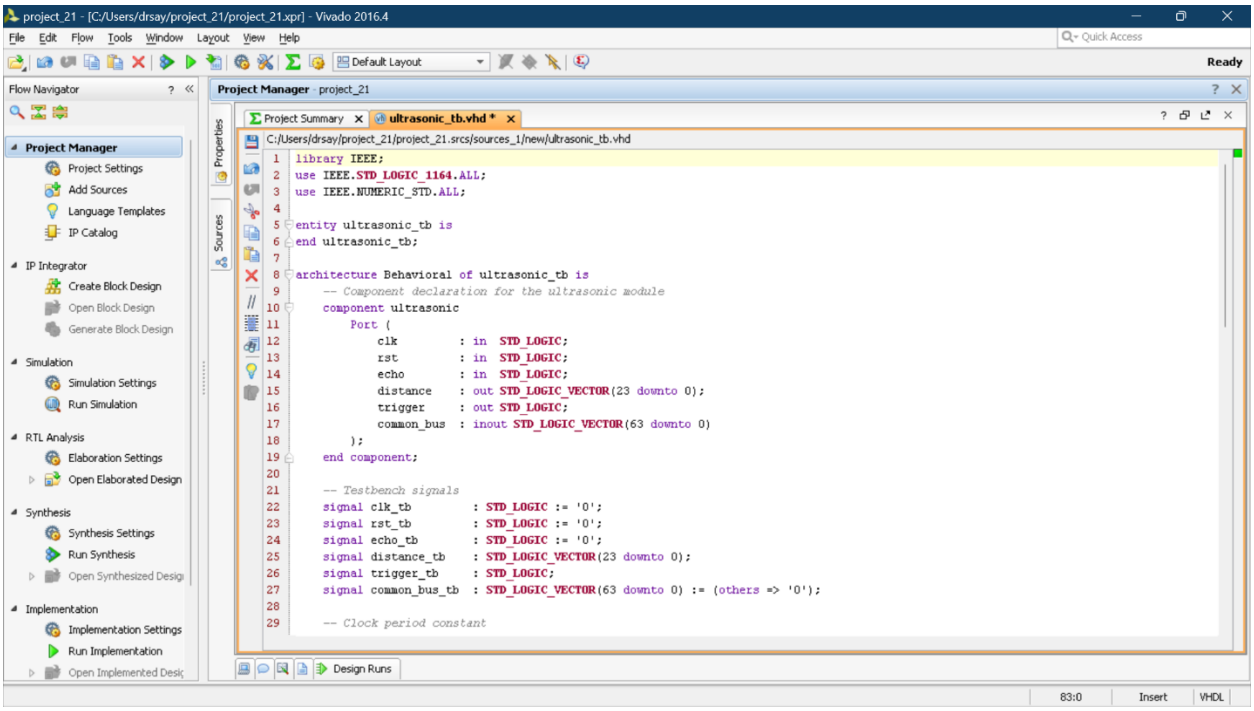

project_22 - [C:/Users/drsay/project_22/project_22.xpr] - Vivado 2016.4
File Edit Flow Tools Window Layout View Help
Quick Access
Default Layout
Ready
Flow Navigator
Project Manager
Project Settings
Add Sources
Language Templates
IP Catalog
IP Integrator
Create Block Design
Open Block Design
Generate Block Design
Simulation
Simulation Settings
Run Simulation
RTL Analysis
Elaboration Settings
Open Elaborated Design
Synthesis
Synthesis Settings
Run Synthesis
Open Synthesized Design
Implementation
Implementation Settings
Run Implementation
Open Implemented Design
Project Manager - project_22
Project Summary
multi_ultrasonic.vhd *
C:/Users/drsay/project_22/project_22.srcs/sources_1/new/multi_ultrasonic.vhd
Properties
Sources
Design Runs
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.NUMERIC_STD.ALL;
4
5 entity multi_ultrasonic is
6 Port (
7 clk : in STD_LOGIC; -- 100 MHz clock
8 rst : in STD_LOGIC; -- Reset signal
9 echo_A1 : in STD_LOGIC; -- Echo signal from sensor A1
10 echo_A2 : in STD_LOGIC; -- Echo signal from sensor A2
11 echo_B1 : in STD_LOGIC; -- Echo signal from sensor B1
12 echo_B2 : in STD_LOGIC; -- Echo signal from sensor B2
13 trigger_A1 : out std_logic;
14 trigger_A2 : out std_logic;
15 trigger_B1 : out std_logic;
16 trigger_B2 : out std_logic;
17 common_bus : inout STD_LOGIC_VECTOR(63 downto 0); -- Shared data bus
18 buzzer : out std_logic -- Buzzer output
19);
20 end multi_ultrasonic;
21
22 architecture Behavioral of multi_ultrasonic is
23 -- Signals for distances
24 signal distance_A1 : STD_LOGIC_VECTOR(23 downto 0);
25 signal distance_A2 : STD_LOGIC_VECTOR(23 downto 0);
26 signal distance_B1 : STD_LOGIC_VECTOR(23 downto 0);
27 signal distance_B2 : STD_LOGIC_VECTOR(23 downto 0);
28 constant delay_ms:integer:=100_000;
29 constant delay_us:integer:=100;


project_22 - [C:/Users/drsay/project_22/project_22.xpr] - Vivado 2016.4
File Edit Flow Tools Window Layout View Help
Quick Access
Default Layout
Ready
Flow Navigator
Project Manager
Project Settings
Add Sources
Language Templates
IP Catalog
IP Integrator
Create Block Design
Open Block Design
Generate Block Design
Simulation
Simulation Settings
Run Simulation
RTL Analysis
Elaboration Settings
Open Elaborated Design
Synthesis
Synthesis Settings
Run Synthesis
Open Synthesized Design
Implementation
Implementation Settings
Run Implementation
Open Implemented Design
Project Manager - project_22
Project Summary
multi_ultrasonic.vhd *
C:/Users/drsay/project_22/project_22.srcs/sources_1/new/multi_ultrasonic.vhd
Properties
Sources
Design Runs
28 constant delay_ms:integer:=100_000;
29 constant delay_us:integer:=100;
30 -- Constants and internal signals
31 signal counter : integer := 0;
32 signal inner_trigger : std_logic;
33 signal trig_counter : integer := 0;
34 constant alert_distance : integer := 5;
35 constant clk_freq : integer := 100_000_000;
36 signal common_bus2 : STD_LOGIC_VECTOR(63 downto 0);
37
38 signal Lane_A_Red : std_logic;
39 signal Lane_B_Red : std_logic;
40
41 -- Component Declaration
42 component ultrasonic is
43 Port (
44 clk : in STD_LOGIC; -- 100 MHz clock
45 rst : in STD_LOGIC; -- Reset signal
46 echo : in STD_LOGIC; -- Echo signal from sensor
47 distance : out STD_LOGIC_VECTOR(23 downto 0); -- Distance output
48 trigger : out std_logic;
49 common_bus : inout STD_LOGIC_VECTOR(63 downto 0) -- Shared data bus
50);
51 end component;
52
53 begin
54 -- Instantiate Ultrasonic Component for A1
55 ultrasonic_A1: ultrasonic
56 port map (

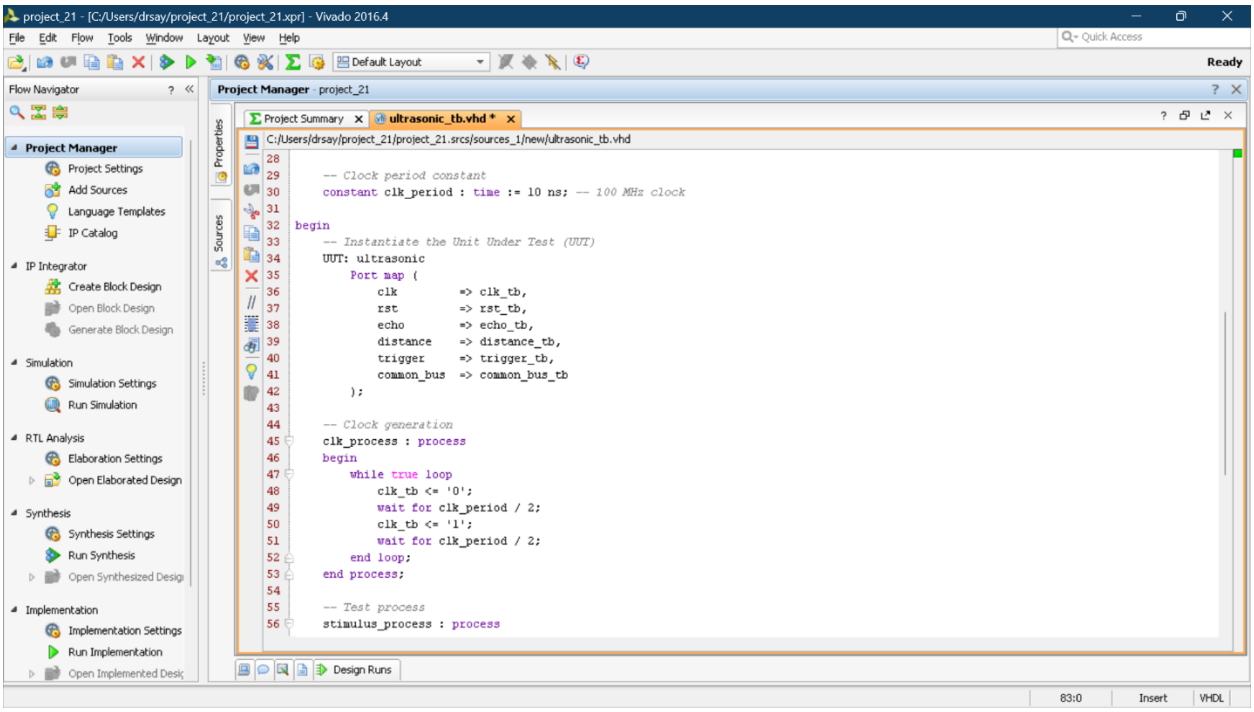

project_22 - [C:/Users/drsay/project_22/project_22.xpr] - Vivado 2016.4
File Edit Flow Tools Window Layout View Help
Quick Access
Default Layout
Ready
Flow Navigator
Project Manager
Project Settings
Add Sources
Language Templates
IP Catalog
IP Integrator
Create Block Design
Open Block Design
Generate Block Design
Simulation
Simulation Settings
Run Simulation
RTL Analysis
Elaboration Settings
Open Elaborated Design
Synthesis
Synthesis Settings
Run Synthesis
Open Synthesized Design
Implementation
Implementation Settings
Run Implementation
Open Implemented Design
Project Manager - project_22
Project Summary
multi_ultrasonic.vhd *
C:/Users/drsay/project_22/project_22.srcs/sources_1/new/multi_ultrasonic.vhd
Properties
Sources
Design Runs
55 ultrasonic_A1: ultrasonic
56 port map (
57 clk => clk,
58 rst => rst,
59 echo => echo_A1,
60 distance => distance_A1,
61 trigger=> trigger_A1,
62 common_bus => common_bus
63);
64
65 -- Instantiate Ultrasonic Component for A2
66 ultrasonic_A2: ultrasonic
67 port map (
68 clk => clk,
69 rst => rst,
70 echo => echo_A2,
71 distance => distance_A2,
72 trigger=> trigger_A2,
73 common_bus => common_bus2
74);
75
76 -- Instantiate Ultrasonic Component for B1
77 ultrasonic_B1: ultrasonic
78 port map (
79 clk => clk,
80 rst => rst,
81 echo => echo_B1,
82 distance => distance_B1,
83 trigger=> trigger_B1,
project_22 - [C:/Users/drsay/project_22/project_22.xpr] - Vivado 2016.4
File Edit Flow Tools Window Layout View Help
Quick Access
Default Layout
Ready
Flow Navigator
Project Manager
Project Settings
Add Sources
Language Templates
IP Catalog
IP Integrator
Create Block Design
Open Block Design
Generate Block Design
Simulation
Simulation Settings
Run Simulation
RTL Analysis
Elaboration Settings
Open Elaborated Design
Synthesis
Synthesis Settings
Run Synthesis
Open Synthesized Design
Implementation
Implementation Settings
Run Implementation
Open Implemented Design
Project Manager - project_22
Project Summary
multi_ultrasonic.vhd *
C:/Users/drsay/project_22/project_22.srcs/sources_1/new/multi_ultrasonic.vhd
Properties
Sources
Design Runs
82 distance => distance_B1,
83 trigger=> trigger_B1,
84 common_bus => common_bus2
85);
86
87 -- Instantiate Ultrasonic Component for B2
88 ultrasonic_B2: ultrasonic
89 port map (
90 clk => clk,
91 rst => rst,
92 echo => echo_B2,
93 distance => distance_B2,
94 trigger=> trigger_B2,
95 common_bus => common_bus2
96);
97
98 -- Distance checking process
99 process (clk)
100 variable distanceA1 : integer;
101 variable distanceA2 : integer;
102 variable distanceB1 : integer;
103 variable distanceB2 : integer;
104 begin
105 if rising_edge(clk) then
106 distanceA1 := to_integer(unsigned(distance_A1));
107 distanceA2 := to_integer(unsigned(distance_A2));
108 distanceB1 := to_integer(unsigned(distance_B1));
109 distanceB2 := to_integer(unsigned(distance_B2));
110 if Lane_A_Red = '1' and Lane_B_Red = '1' then



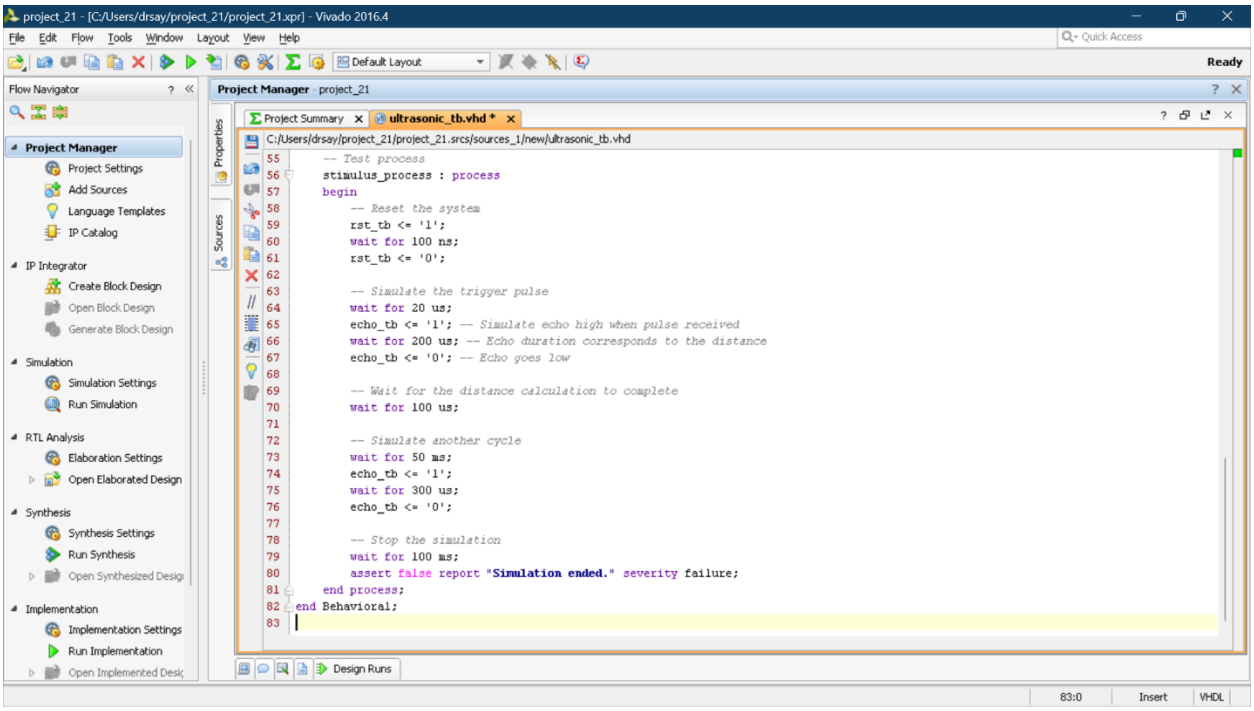

```
109             distanceB2 := to_integer(unsigned(distance_B2));
110             if Lane_A_Red = '1' and Lane_B_Red = '1' then
111                 common_bus(45) <= '0';
112             elsif Lane_A_Red = '1' and (distanceA1 < alert_distance or distanceA2 < alert_distance) then
113                 common_bus(45) <= '1';
114             elsif Lane_B_Red = '1' and (distanceB1 < alert_distance or distanceB2 < alert_distance) then
115                 common_bus(45) <= '1';
116             else
117                 common_bus(45) <= '0';
118             end if;
119         end if;
120     end process;
121
122     -- Buzzer control
123     process (clk)
124     begin
125         if rising_edge(clk) then
126             if common_bus(45)='1' then   -- 2 seconds
127                 buzzer <= '1';
128             else
129                 buzzer <= '0';
130             end if;
131         end if;
132     end process;
133
134     -- Assign traffic-related signals
135     Lane_A_Red <= common_bus(47);
136     Lane_B_Red <= common_bus(46);
137
```



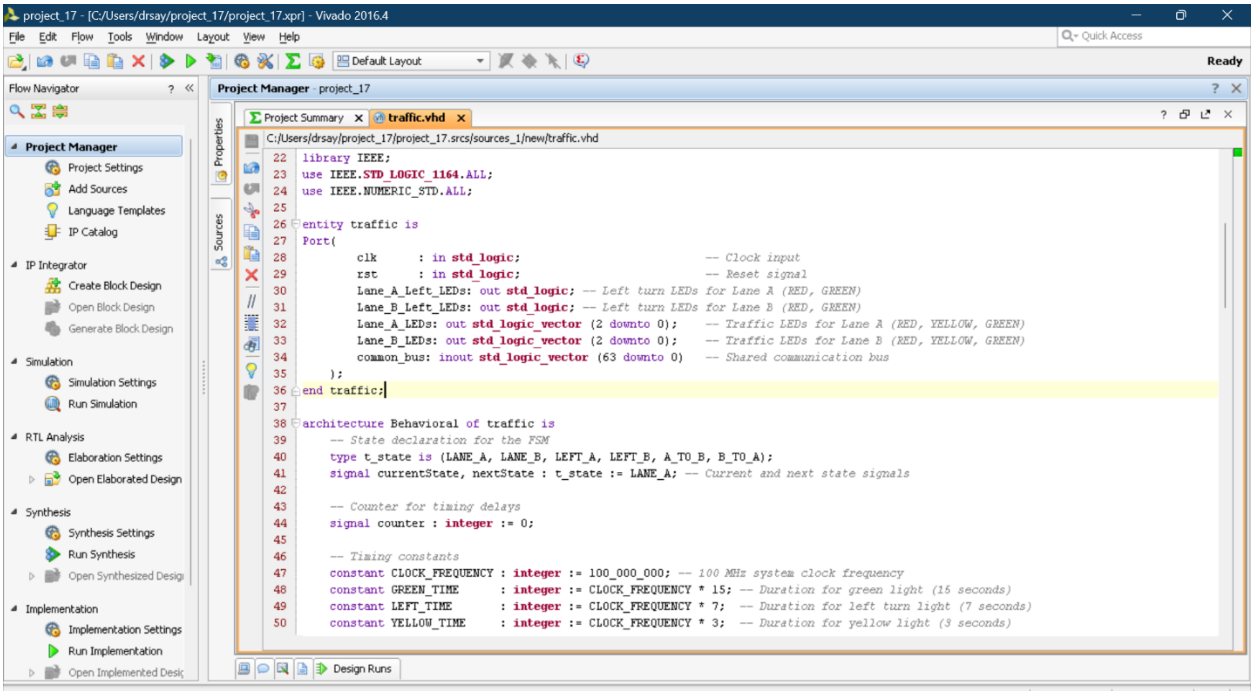

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity ultrasonic_tb is
6  end ultrasonic_tb;
7
8  architecture Behavioral of ultrasonic_tb is
9      -- Component declaration for the ultrasonic module
10     component ultrasonic
11         Port (
12             clk         : in  STD_LOGIC;
13             rst         : in  STD_LOGIC;
14             echo        : in  STD_LOGIC;
15             distance    : out STD_LOGIC_VECTOR(23 downto 0);
16             trigger     : out STD_LOGIC;
17             common_bus  : inout STD_LOGIC_VECTOR(63 downto 0)
18         );
19     end component;
20
21     -- Testbench signals
22     signal clk_tb         : STD_LOGIC := '0';
23     signal rst_tb         : STD_LOGIC := '0';
24     signal echo_tb        : STD_LOGIC := '0';
25     signal distance_tb    : STD_LOGIC_VECTOR(23 downto 0);
26     signal trigger_tb     : STD_LOGIC;
27     signal common_bus_tb  : STD_LOGIC_VECTOR(63 downto 0) := (others => '0');
28
29     -- Clock period constant
```



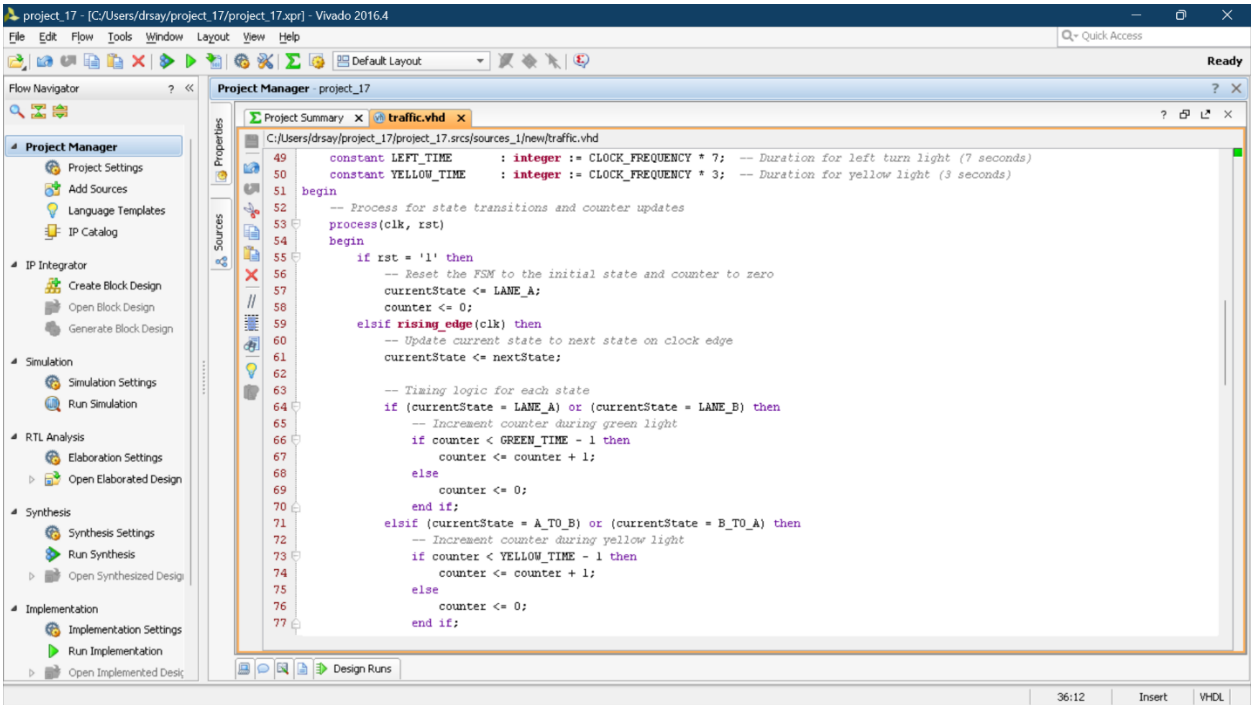

```
28
29     -- Clock period constant
30     constant clk_period : time := 10 ns; -- 100 MHz clock
31
32 begin
33     -- Instantiate the Unit Under Test (UUT)
34     UUT: ultrasonic
35         Port map (
36             clk         => clk_tb,
37             rst         => rst_tb,
38             echo        => echo_tb,
39             distance    => distance_tb,
40             trigger     => trigger_tb,
41             common_bus  => common_bus_tb
42         );
43
44     -- Clock generation
45     clk_process : process
46     begin
47         while true loop
48             clk_tb <= '0';
49             wait for clk_period / 2;
50             clk_tb <= '1';
51             wait for clk_period / 2;
52         end loop;
53     end process;
54
55     -- Test process
56     stimulus_process : process
```


project_21 - [C:/Users/drsay/project_21/project_21.xpr] - Vivado 2016.4
File Edit Flow Tools Window Layout View Help
Quick Access
Default Layout
Ready
Flow Navigator
Project Manager
Project Settings
Add Sources
Language Templates
IP Catalog
IP Integrator
Create Block Design
Open Block Design
Generate Block Design
Simulation
Simulation Settings
Run Simulation
RTL Analysis
Elaboration Settings
Open Elaborated Design
Synthesis
Synthesis Settings
Run Synthesis
Open Synthesized Desig
Implementation
Implementation Settings
Run Implementation
Open Implemented Desi
Project Manager project_21
Project Summary
ultrasonic_tb.vhd *
C:/Users/drsay/project_21/project_21.srcs/sources_1/new/ultrasonic_tb.vhd
Properties
Sources
55 -- Test process
56 stimulus_process : process
57 begin
58 -- Reset the system
59 rst_tb <= '1';
60 wait for 100 ns;
61 rst_tb <= '0';
62
63 -- Simulate the trigger pulse
64 wait for 20 us;
65 echo_tb <= '1'; -- Simulate echo high when pulse received
66 wait for 200 us; -- Echo duration corresponds to the distance
67 echo_tb <= '0'; -- Echo goes low
68
69 -- Wait for the distance calculation to complete
70 wait for 100 us;
71
72 -- Simulate another cycle
73 wait for 50 ms;
74 echo_tb <= '1';
75 wait for 300 us;
76 echo_tb <= '0';
77
78 -- Stop the simulation
79 wait for 100 ms;
80 assert false report "Simulation ended." severity failure;
81 end process;
82 end Behavioral;
83
Design Runs
83:0
Insert
VHDL

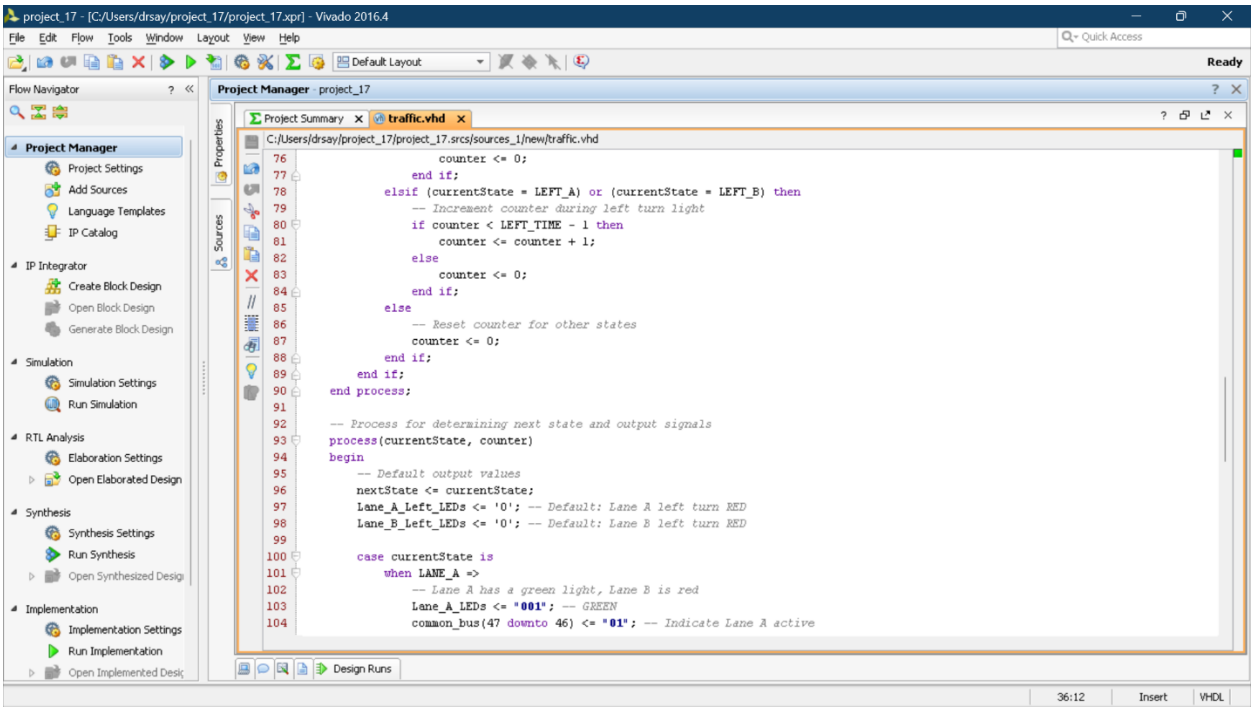
For Traffic Lights:



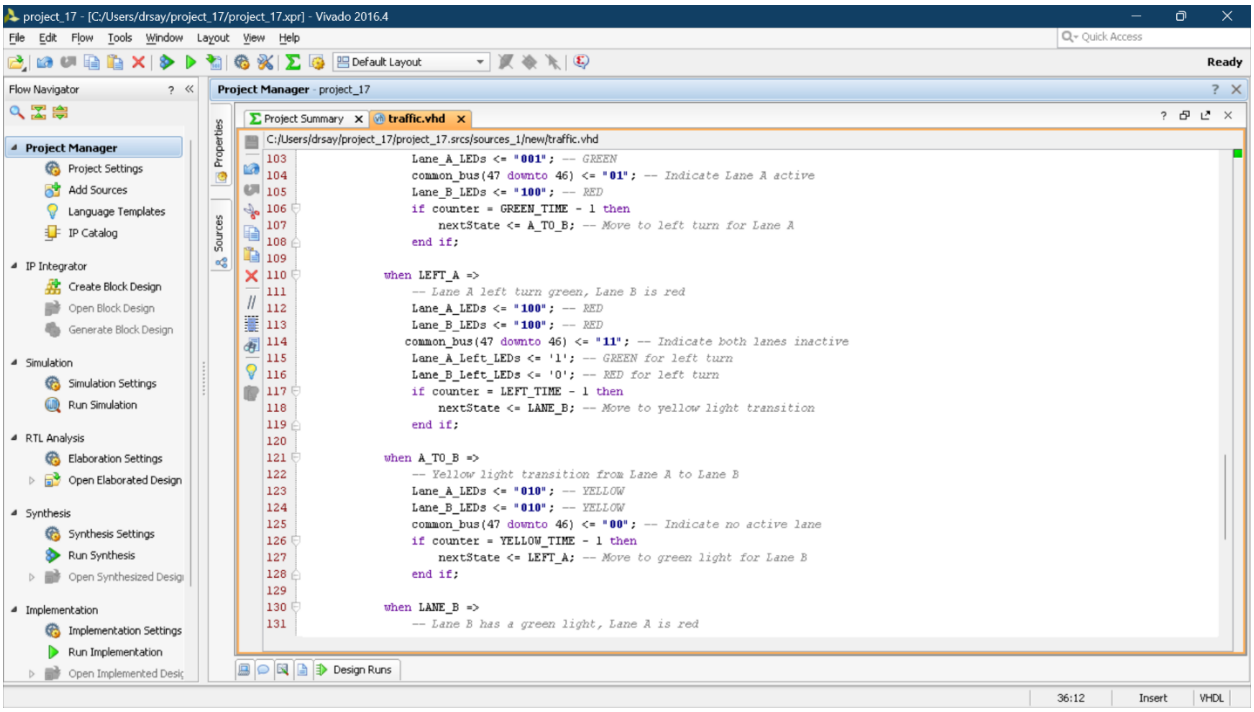

```
22  library IEEE;
23  use IEEE.STD_LOGIC_1164.ALL;
24  use IEEE.NUMERIC_STD.ALL;
25
26  entity traffic is
27  Port(
28          clk      : in std_logic;                          -- Clock input
29          rst      : in std_logic;                          -- Reset signal
30          Lane_A_Left_LEDs: out std_logic; -- Left turn LEDs for Lane A (RED, GREEN)
31          Lane_B_Left_LEDs: out std_logic; -- Left turn LEDs for Lane B (RED, GREEN)
32          Lane_A_LEDs: out std_logic_vector (2 downto 0);    -- Traffic LEDs for Lane A (RED, YELLOW, GREEN)
33          Lane_B_LEDs: out std_logic_vector (2 downto 0);    -- Traffic LEDs for Lane B (RED, YELLOW, GREEN)
34          common_bus: inout std_logic_vector (63 downto 0)   -- Shared communication bus
35      );
36  end traffic;
37
38  architecture Behavioral of traffic is
39      -- State declaration for the FSM
40      type t_state is (LANE_A, LANE_B, LEFT_A, LEFT_B, A_TO_B, B_TO_A);
41      signal currentState, nextState : t_state := LANE_A; -- Current and next state signals
42
43      -- Counter for timing delays
44      signal counter : integer := 0;
45
46      -- Timing constants
47      constant CLOCK_FREQUENCY : integer := 100_000_000; -- 100 MHz system clock frequency
48      constant GREEN_TIME      : integer := CLOCK_FREQUENCY * 15;  -- Duration for green light (15 seconds)
49      constant LEFT_TIME       : integer := CLOCK_FREQUENCY * 7;   -- Duration for left turn light (7 seconds)
50      constant YELLOW_TIME     : integer := CLOCK_FREQUENCY * 3;   -- Duration for yellow light (3 seconds)
```



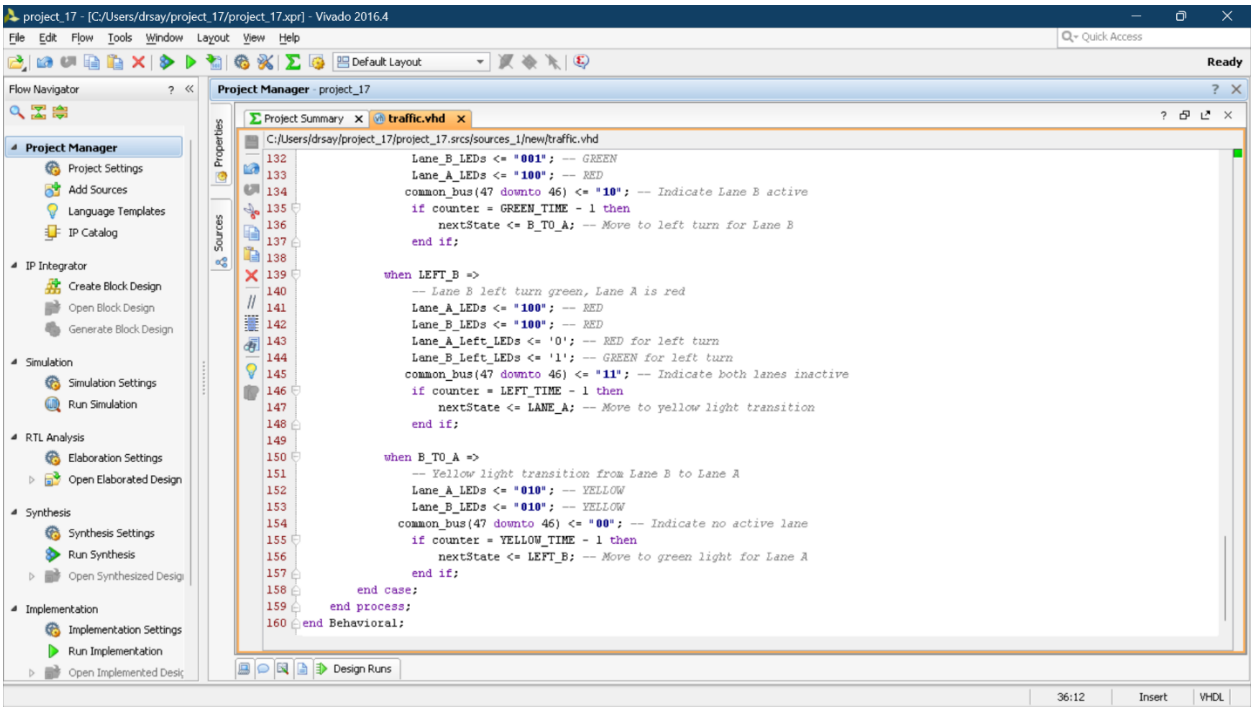

```
49      constant LEFT_TIME       : integer := CLOCK_FREQUENCY * 7;  -- Duration for left turn light (7 seconds)
50      constant YELLOW_TIME     : integer := CLOCK_FREQUENCY * 3;  -- Duration for yellow light (3 seconds)
51  begin
52      -- Process for state transitions and counter updates
53      process(clk, rst)
54      begin
55          if rst = '1' then
56              -- Reset the FSM to the initial state and counter to zero
57              currentState <= LANE_A;
58              counter <= 0;
59          elsif rising_edge(clk) then
60              -- Update current state to next state on clock edge
61              currentState <= nextState;
62
63              -- Timing logic for each state
64              if (currentState = LANE_A) or (currentState = LANE_B) then
65                  -- Increment counter during green light
66                  if counter < GREEN_TIME - 1 then
67                      counter <= counter + 1;
68                  else
69                      counter <= 0;
70                  end if;
71              elsif (currentState = A_TO_B) or (currentState = B_TO_A) then
72                  -- Increment counter during yellow light
73                  if counter < YELLOW_TIME - 1 then
74                      counter <= counter + 1;
75                  else
76                      counter <= 0;
77                  end if;
```



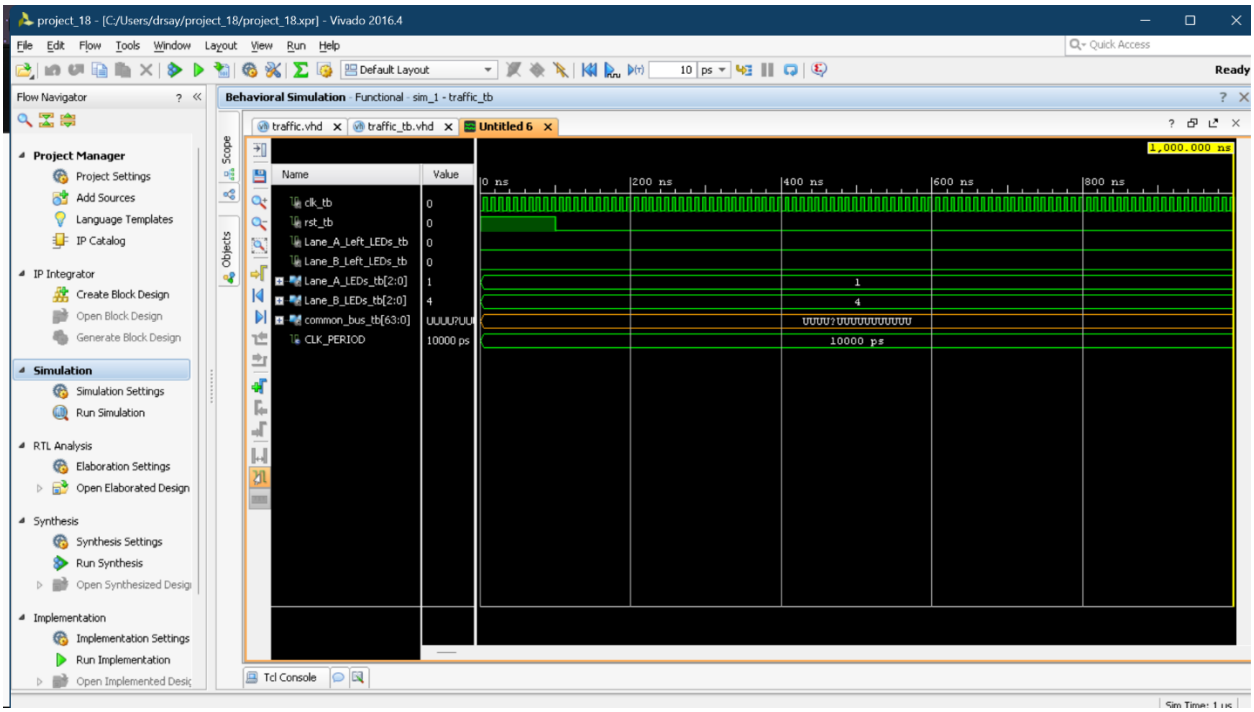

```
76                  counter <= 0;
77              end if;
78          elsif (currentState = LEFT_A) or (currentState = LEFT_B) then
79              -- Increment counter during left turn light
80              if counter < LEFT_TIME - 1 then
81                  counter <= counter + 1;
82              else
83                  counter <= 0;
84              end if;
85          else
86              -- Reset counter for other states
87              counter <= 0;
88          end if;
89      end if;
90  end process;
91
92  -- Process for determining next state and output signals
93  process(currentState, counter)
94  begin
95      -- Default output values
96      nextState <= currentState;
97      Lane_A_Left_LEDs <= '0'; -- Default: Lane A left turn RED
98      Lane_B_Left_LEDs <= '0'; -- Default: Lane B left turn RED
99
100     case currentState is
101         when LANE_A =>
102             -- Lane A has a green light, Lane B is red
103             Lane_A_LEDs <= "001"; -- GREEN
104             common_bus(47 downto 46) <= "01"; -- Indicate Lane A active
```

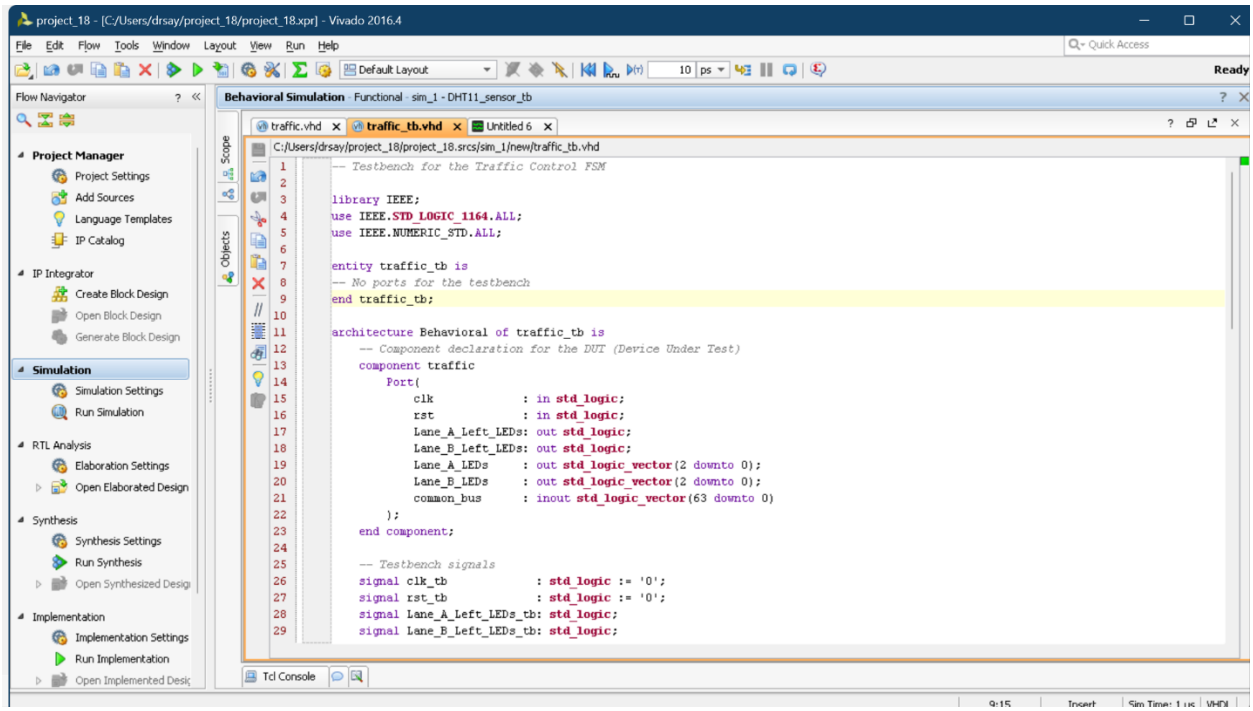


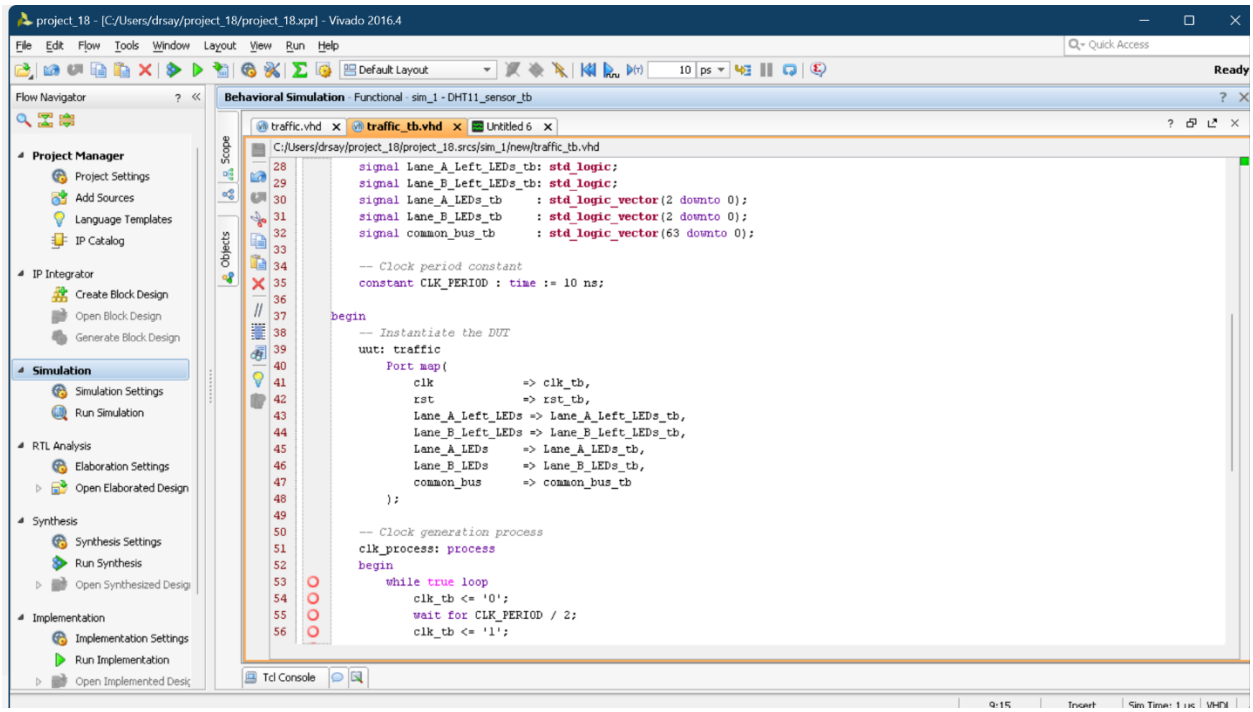

```
103                 Lane_A_LEDs <= "001"; -- GREEN
104                 common_bus(47 downto 46) <= "01"; -- Indicate Lane A active
105                 Lane_B_LEDs <= "100"; -- RED
106                 if counter = GREEN_TIME - 1 then
107                     nextState <= A_TO_B; -- Move to left turn for Lane A
108                 end if;
109
110             when LEFT_A =>
111                 -- Lane A left turn green, Lane B is red
112                 Lane_A_LEDs <= "100"; -- RED
113                 Lane_B_LEDs <= "100"; -- RED
114                 common_bus(47 downto 46) <= "11"; -- Indicate both lanes inactive
115                 Lane_A_Left_LEDs <= '1'; -- GREEN for left turn
116                 Lane_B_Left_LEDs <= '0'; -- RED for left turn
117                 if counter = LEFT_TIME - 1 then
118                     nextState <= LANE_B; -- Move to yellow light transition
119                 end if;
120
121             when A_TO_B =>
122                 -- Yellow light transition from Lane A to Lane B
123                 Lane_A_LEDs <= "010"; -- YELLOW
124                 Lane_B_LEDs <= "010"; -- YELLOW
125                 common_bus(47 downto 46) <= "00"; -- Indicate no active lane
126                 if counter = YELLOW_TIME - 1 then
127                     nextState <= LEFT_A; -- Move to green light for Lane B
128                 end if;
129
130             when LANE_B =>
131                 -- Lane B has a green light, Lane A is red
```

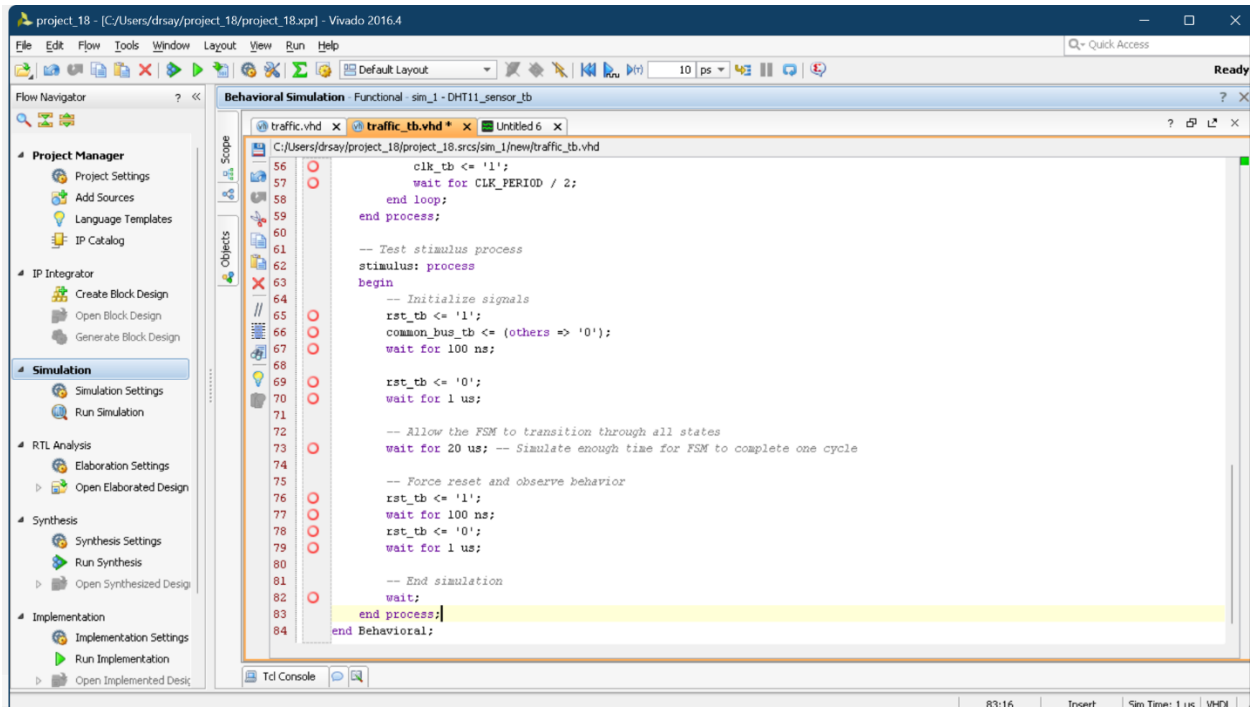



```
132                 Lane_B_LEDs <= "001"; -- GREEN
133                 Lane_A_LEDs <= "100"; -- RED
134                 common_bus(47 downto 46) <= "10"; -- Indicate Lane B active
135                 if counter = GREEN_TIME - 1 then
136                     nextState <= B_TO_A; -- Move to left turn for Lane B
137                 end if;
138
139             when LEFT_B =>
140                 -- Lane B left turn green, Lane A is red
141                 Lane_A_LEDs <= "100"; -- RED
142                 Lane_B_LEDs <= "100"; -- RED
143                 Lane_A_Left_LEDs <= '0'; -- RED for left turn
144                 Lane_B_Left_LEDs <= '1'; -- GREEN for left turn
145                 common_bus(47 downto 46) <= "11"; -- Indicate both lanes inactive
146                 if counter = LEFT_TIME - 1 then
147                     nextState <= LANE_A; -- Move to yellow light transition
148                 end if;
149
150             when B_TO_A =>
151                 -- Yellow light transition from Lane B to Lane A
152                 Lane_A_LEDs <= "010"; -- YELLOW
153                 Lane_B_LEDs <= "010"; -- YELLOW
154                 common_bus(47 downto 46) <= "00"; -- Indicate no active lane
155                 if counter = YELLOW_TIME - 1 then
156                     nextState <= LEFT_B; -- Move to green light for Lane A
157                 end if;
158         end case;
159     end process;
160 end Behavioral;
```

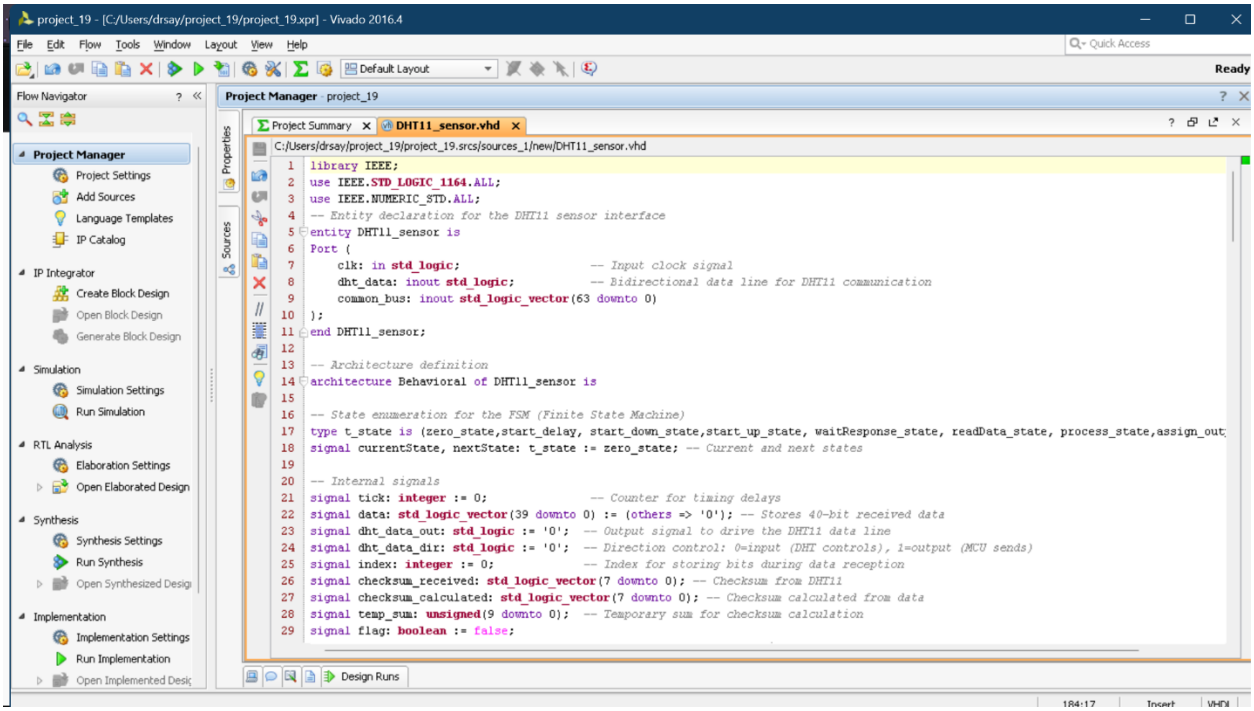


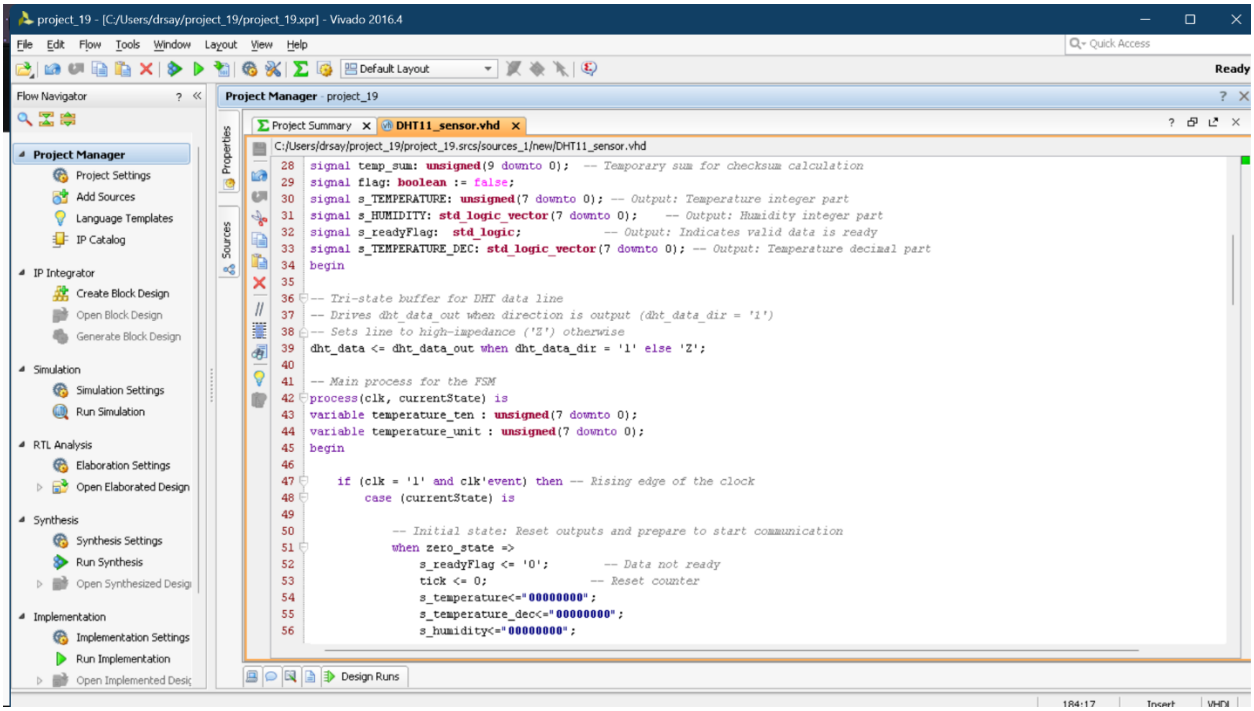

project_18 - [C:/Users/drsay/project_18/project_18.xpr] - Vivado 2016.4
File Edit Flow Tools Window Layout View Run Help
Quick Access
Default Layout
10 ps
Ready
Flow Navigator
Project Manager
Project Settings
Add Sources
Language Templates
IP Catalog
IP Integrator
Create Block Design
Open Block Design
Generate Block Design
Simulation
Simulation Settings
Run Simulation
RTL Analysis
Elaboration Settings
Open Elaborated Design
Synthesis
Synthesis Settings
Run Synthesis
Open Synthesized Design
Implementation
Implementation Settings
Run Implementation
Open Implemented Design
Behavioral Simulation - Functional - sim_1 - DHT11_sensor_tb
traffic.vhd
traffic_tb.vhd
Untitled 6
C:/Users/drsay/project_18/project_18.srcs/sim_1/new/traffic_tb.vhd
Scope
Objects
Tcl Console
1 -- Testbench for the Traffic Control FSM
2
3 library IEEE;
4 use IEEE.STD_LOGIC_1164.ALL;
5 use IEEE.NUMERIC_STD.ALL;
6
7 entity traffic_tb is
8 -- No ports for the testbench
9 end traffic_tb;
10
11 architecture Behavioral of traffic_tb is
12 -- Component declaration for the DUT (Device Under Test)
13 component traffic
14 Port(
15 clk : in std_logic;
16 rst : in std_logic;
17 Lane_A_Left_LEDs: out std_logic;
18 Lane_B_Left_LEDs: out std_logic;
19 Lane_A_LEDs : out std_logic_vector(2 downto 0);
20 Lane_B_LEDs : out std_logic_vector(2 downto 0);
21 common_bus : inout std_logic_vector(63 downto 0)
22);
23 end component;
24
25 -- Testbench signals
26 signal clk_tb : std_logic := '0';
27 signal rst_tb : std_logic := '0';
28 signal Lane_A_Left_LEDs_tb: std_logic;
29 signal Lane_B_Left_LEDs_tb: std_logic;

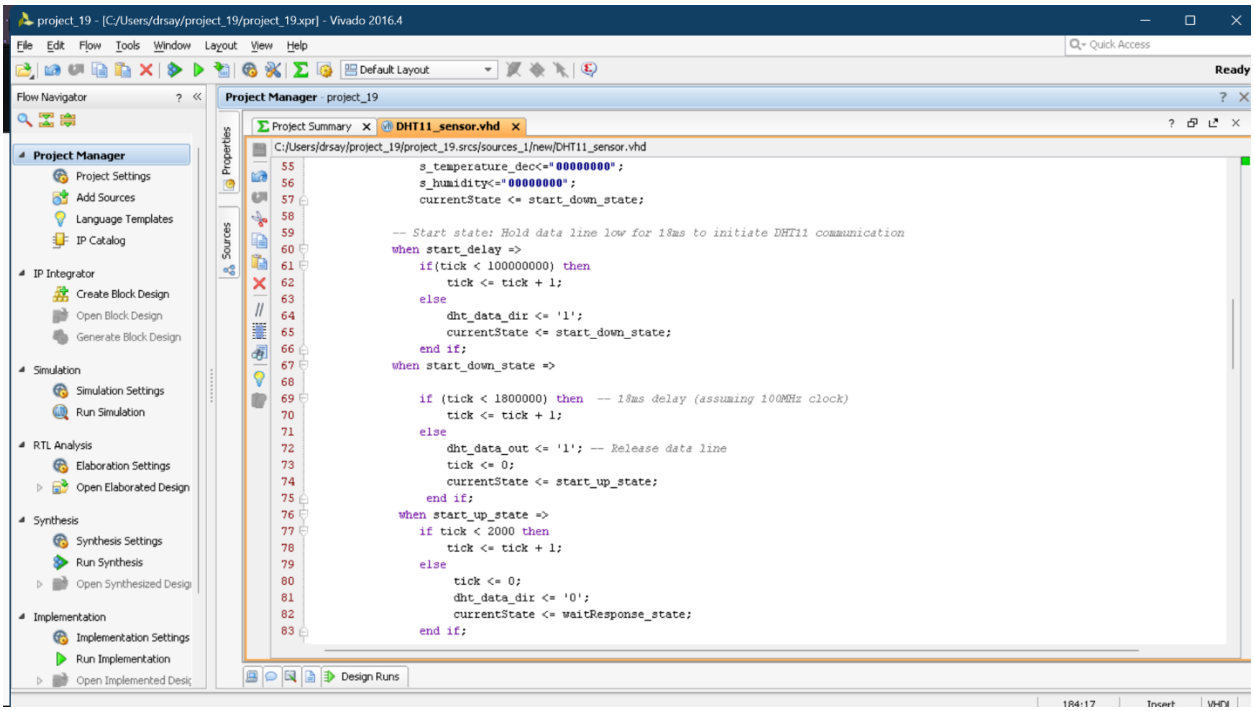

project_18 - [C:/Users/drsay/project_18/project_18.xpr] - Vivado 2016.4
File Edit Flow Tools Window Layout View Run Help
Quick Access
Default Layout
10 ps
Ready
Flow Navigator
Project Manager
Project Settings
Add Sources
Language Templates
IP Catalog
IP Integrator
Create Block Design
Open Block Design
Generate Block Design
Simulation
Simulation Settings
Run Simulation
RTL Analysis
Elaboration Settings
Open Elaborated Design
Synthesis
Synthesis Settings
Run Synthesis
Open Synthesized Design
Implementation
Implementation Settings
Run Implementation
Open Implemented Design
Behavioral Simulation - Functional - sim_1 - DHT11_sensor_tb
traffic.vhd
traffic_tb.vhd
Untitled 6
C:/Users/drsay/project_18/project_18.srcs/sim_1/new/traffic_tb.vhd
Scope
Objects
Tcl Console
28 signal Lane_A_Left_LEDs_tb: std_logic;
29 signal Lane_B_Left_LEDs_tb: std_logic;
30 signal Lane_A_LEDs_tb : std_logic_vector(2 downto 0);
31 signal Lane_B_LEDs_tb : std_logic_vector(2 downto 0);
32 signal common_bus_tb : std_logic_vector(63 downto 0);
33
34 -- Clock period constant
35 constant CLK_PERIOD : time := 10 ns;
36
37 begin
38 -- Instantiate the DUT
39 uut: traffic
40 Port map(
41 clk => clk_tb,
42 rst => rst_tb,
43 Lane_A_Left_LEDs => Lane_A_Left_LEDs_tb,
44 Lane_B_Left_LEDs => Lane_B_Left_LEDs_tb,
45 Lane_A_LEDs => Lane_A_LEDs_tb,
46 Lane_B_LEDs => Lane_B_LEDs_tb,
47 common_bus => common_bus_tb
48);
49
50 -- Clock generation process
51 clk_process: process
52 begin
53 while true loop
54 clk_tb <= '0';
55 wait for CLK_PERIOD / 2;
56 clk_tb <= '1';

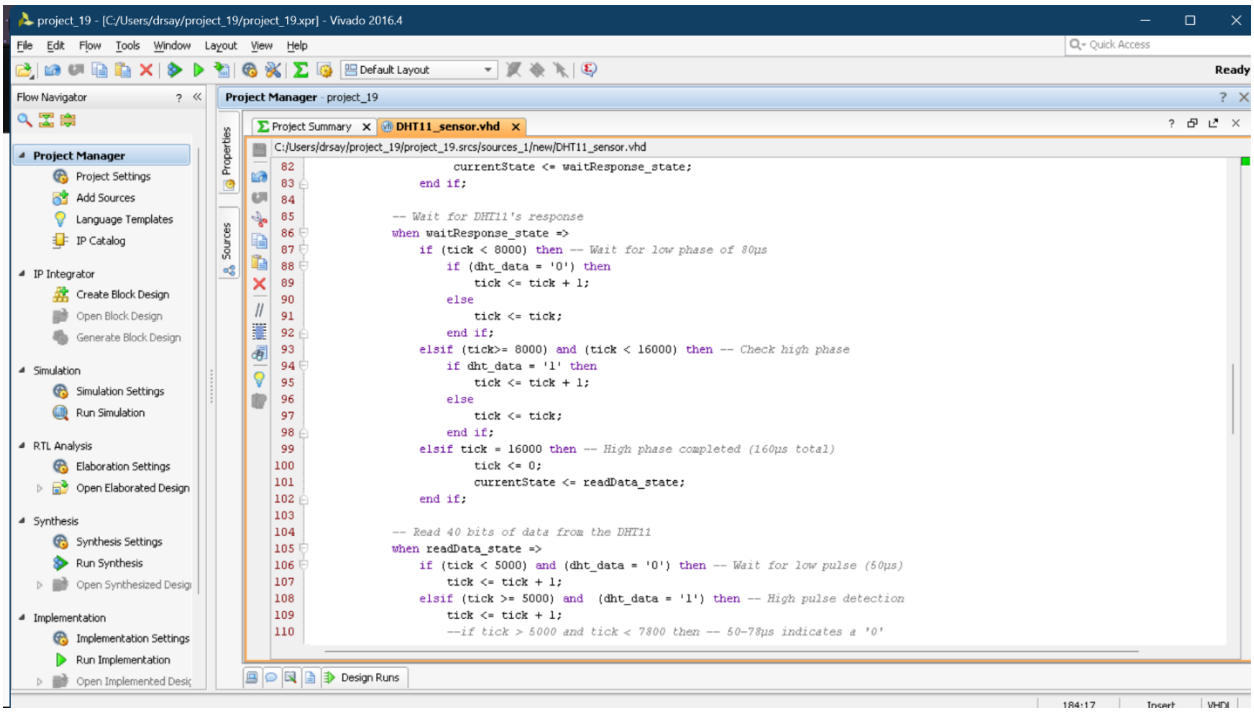

project_18 - [C:/Users/drsay/project_18/project_18.xpr] - Vivado 2016.4
File Edit Flow Tools Window Layout View Run Help
Quick Access
Default Layout
10 ps
Ready
Flow Navigator
Project Manager
Project Settings
Add Sources
Language Templates
IP Catalog
IP Integrator
Create Block Design
Open Block Design
Generate Block Design
Simulation
Simulation Settings
Run Simulation
RTL Analysis
Elaboration Settings
Open Elaborated Design
Synthesis
Synthesis Settings
Run Synthesis
Open Synthesized Design
Implementation
Implementation Settings
Run Implementation
Open Implemented Design
Behavioral Simulation - Functional - sim_1 - DHT11_sensor_tb
Scope
Objects
traffic.vhd
traffic_tb.vhd *
Untitled 6
C:/Users/drsay/project_18/project_18.srcs/sim_1/new/traffic_tb.vhd
clk_tb <= '1';
wait for CLK_PERIOD / 2;
end loop;
end process;
-- Test stimulus process
stimulus: process
begin
-- Initialize signals
rst_tb <= '1';
common_bus_tb <= (others => '0');
wait for 100 ns;
rst_tb <= '0';
wait for 1 us;
-- Allow the FSM to transition through all states
wait for 20 us; -- Simulate enough time for FSM to complete one cycle
-- Force reset and observe behavior
rst_tb <= '1';
wait for 100 ns;
rst_tb <= '0';
wait for 1 us;
-- End simulation
wait;
end process;
end Behavioral;
Tcl Console

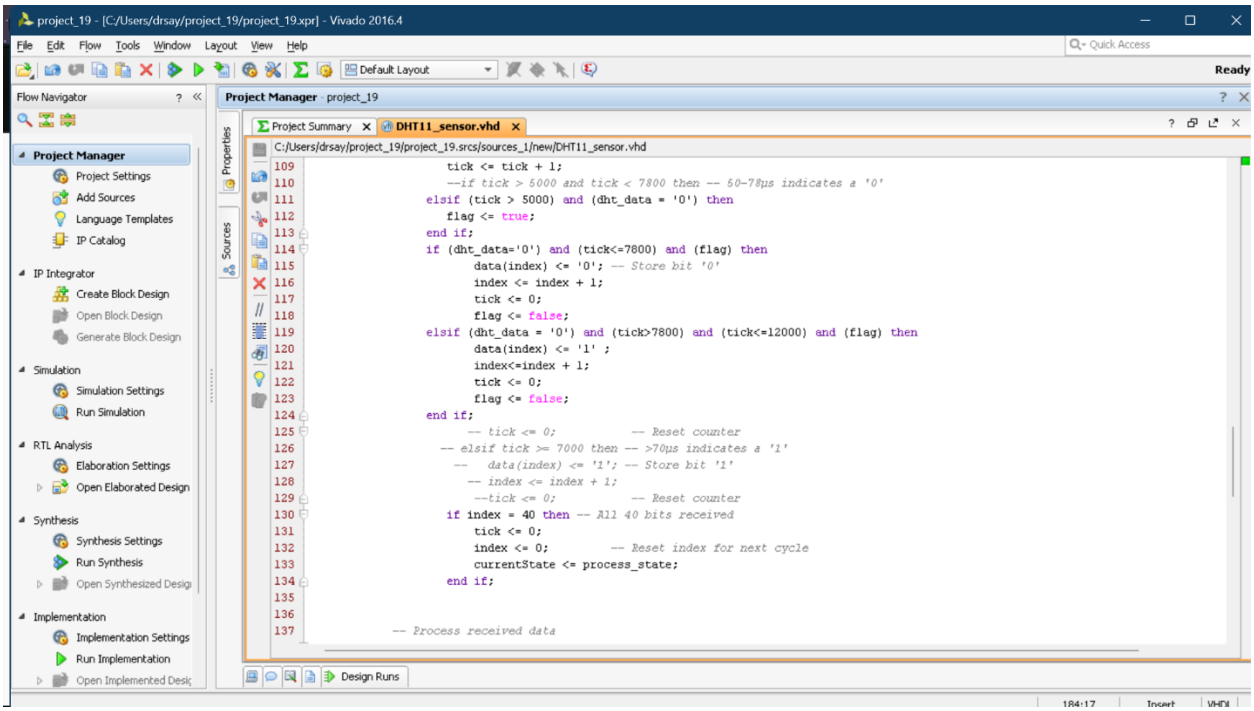
For Temperature Sensor:



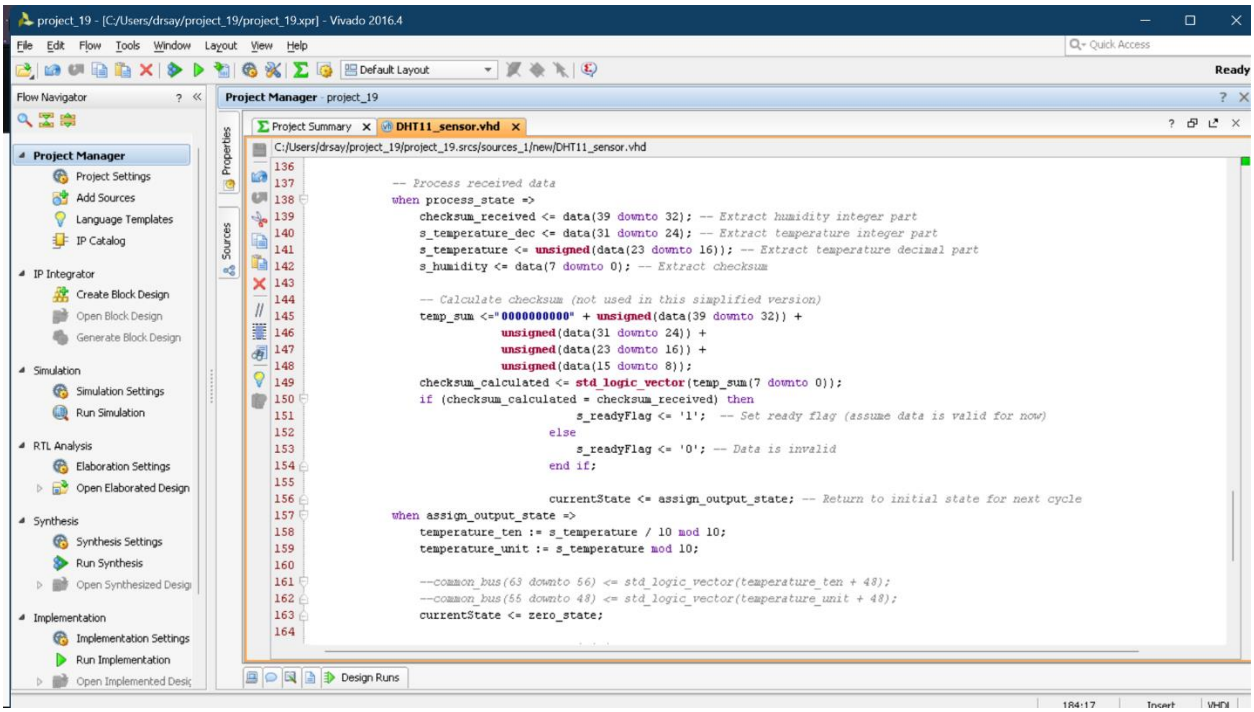



project_19 - [C:/Users/drsay/project_19/project_19.xpr] - Vivado 2016.4
File Edit Flow Tools Window Layout View Help
Quick Access
Default Layout
Ready
Flow Navigator
Project Manager
Project Settings
Add Sources
Language Templates
IP Catalog
IP Integrator
Create Block Design
Open Block Design
Generate Block Design
Simulation
Simulation Settings
Run Simulation
RTL Analysis
Elaboration Settings
Open Elaborated Design
Synthesis
Synthesis Settings
Run Synthesis
Open Synthesized Design
Implementation
Implementation Settings
Run Implementation
Open Implemented Design
Project Manager - project_19
Project Summary
DHT11_sensor.vhd
C:/Users/drsay/project_19/project_19.srcs/sources_1/new/DHT11_sensor.vhd
Properties
Sources
Design Runs
s_temperature_dec<="00000000";
s_humidity<="00000000";
currentState <= start_down_state;
-- Start state: Hold data line low for 18ms to initiate DHT11 communication
when start_delay =>
if(tick < 100000000) then
tick <= tick + 1;
else
dht_data_dir <= '1';
currentState <= start_down_state;
end if;
when start_down_state =>
if (tick < 1800000) then -- 18ms delay (assuming 100MHz clock)
tick <= tick + 1;
else
dht_data_out <= '1'; -- Release data line
tick <= 0;
currentState <= start_up_state;
end if;
when start_up_state =>
if tick < 2000 then
tick <= tick + 1;
else
tick <= 0;
dht_data_dir <= '0';
currentState <= waitResponse_state;
end if;

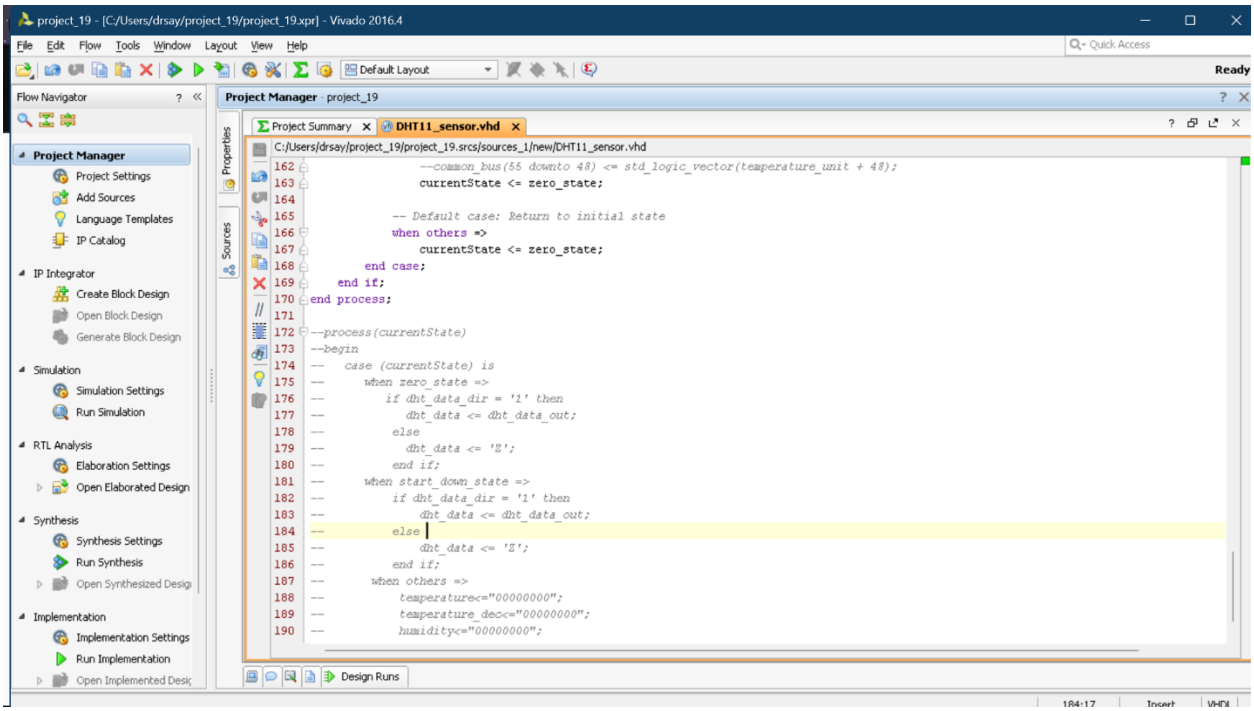

project_19 - [C:/Users/drsay/project_19/project_19.xpr] - Vivado 2016.4
File Edit Flow Tools Window Layout View Help
Quick Access
Default Layout
Ready
Flow Navigator
Project Manager
Project Settings
Add Sources
Language Templates
IP Catalog
IP Integrator
Create Block Design
Open Block Design
Generate Block Design
Simulation
Simulation Settings
Run Simulation
RTL Analysis
Elaboration Settings
Open Elaborated Design
Synthesis
Synthesis Settings
Run Synthesis
Open Synthesized Design
Implementation
Implementation Settings
Run Implementation
Open Implemented Design
Project Manager - project_19
Project Summary
DHT11_sensor.vhd
C:/Users/drsay/project_19/project_19.srcs/sources_1/new/DHT11_sensor.vhd
Properties
Sources
Design Runs
currentState <= waitResponse_state;
end if;
-- Wait for DHT11's response
when waitResponse_state =>
if (tick < 8000) then -- Wait for low phase of 80µs
if (dht_data = '0') then
tick <= tick + 1;
else
tick <= tick;
end if;
elsif (tick>= 8000) and (tick < 16000) then -- Check high phase
if dht_data = '1' then
tick <= tick + 1;
else
tick <= tick;
end if;
elsif tick = 16000 then -- High phase completed (160µs total)
tick <= 0;
currentState <= readData_state;
end if;
-- Read 40 bits of data from the DHT11
when readData_state =>
if (tick < 5000) and (dht_data = '0') then -- Wait for low pulse (50µs)
tick <= tick + 1;
elsif (tick >= 5000) and (dht_data = '1') then -- High pulse detection
tick <= tick + 1;
--if tick > 5000 and tick < 7800 then -- 50-78µs indicates a '0'



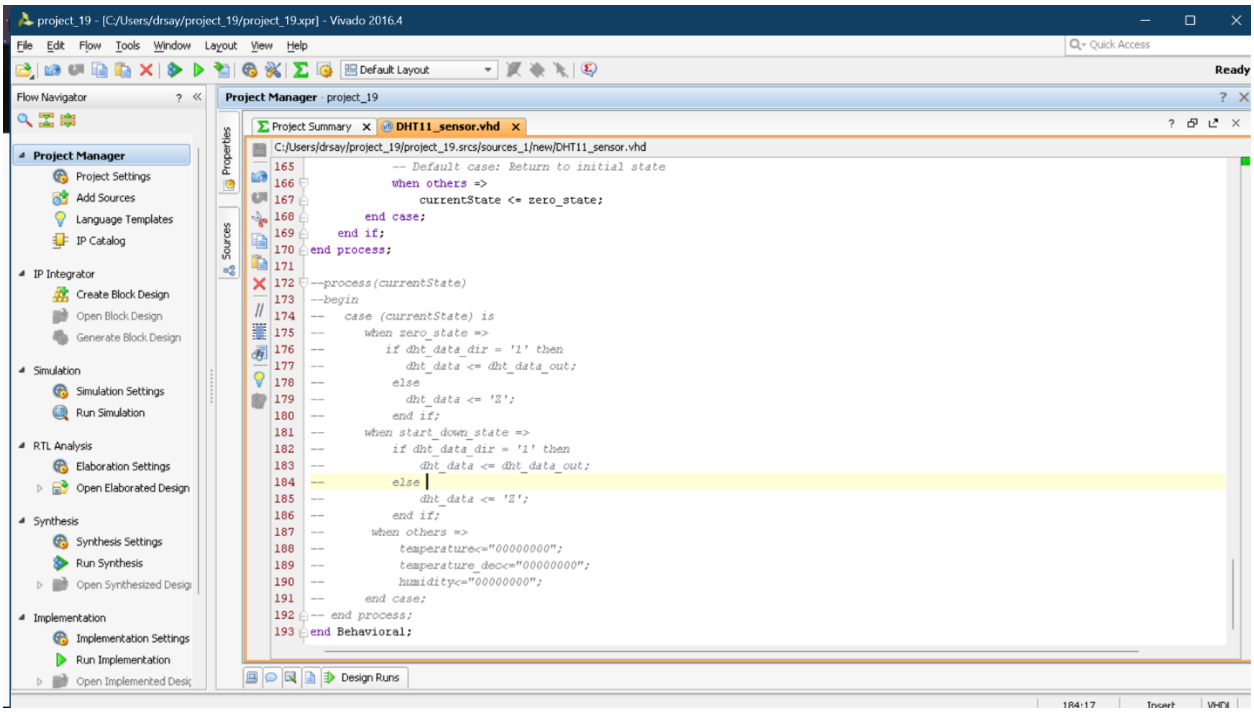

```
109                     tick <= tick + 1;
110                     --if tick > 5000 and tick < 7800 then -- 50-78µs indicates a '0'
111                 elsif (tick > 5000) and (dht_data = '0') then
112                     flag <= true;
113                 end if;
114                 if (dht_data='0') and (tick<=7800) and (flag) then
115                       data(index) <= '0'; -- Store bit '0'
116                       index <= index + 1;
117                       tick <= 0;
118                       flag <= false;
119                 elsif (dht_data = '0') and (tick>7800) and (tick<=12000) and (flag) then
120                       data(index) <= '1' ;
121                       index<=index + 1;
122                       tick <= 0;
123                       flag <= false;
124                 end if;
125                       -- tick <= 0;           -- Reset counter
126                   -- elsif tick >= 7000 then -- >70µs indicates a '1'
127                     --   data(index) <= '1'; -- Store bit '1'
128                       -- index <= index + 1;
129                        --tick <= 0;           -- Reset counter
130                    if index = 40 then -- All 40 bits received
131                       tick <= 0;
132                       index <= 0;          -- Reset index for next cycle
133                       currentState <= process_state;
134                    end if;
135
136
137             -- Process received data
```



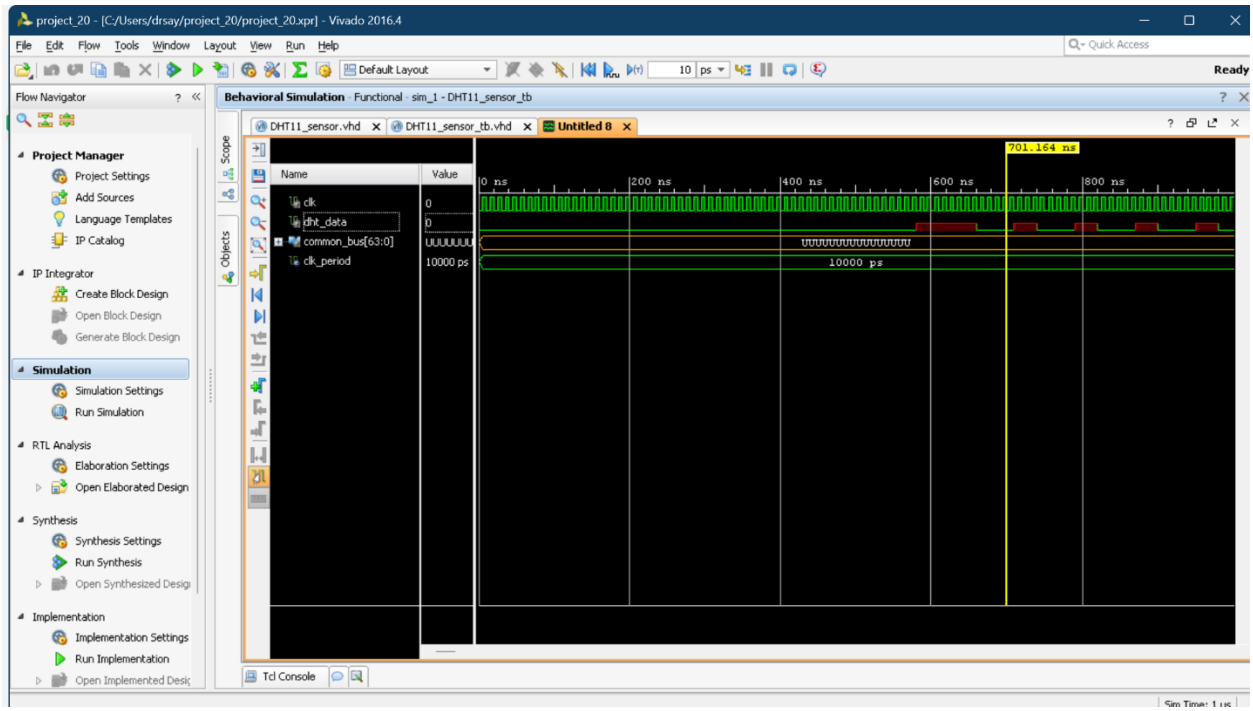

```
136
137             -- Process received data
138             when process_state =>
139                 checksum_received <= data(39 downto 32); -- Extract humidity integer part
140                 s_temperature_dec <= data(31 downto 24); -- Extract temperature integer part
141                 s_temperature <= unsigned(data(23 downto 16)); -- Extract temperature decimal part
142                 s_humidity <= data(7 downto 0); -- Extract checksum
143
144                 -- Calculate checksum (not used in this simplified version)
145                 temp_sum <="0000000000" + unsigned(data(39 downto 32)) +
146                            unsigned(data(31 downto 24)) +
147                            unsigned(data(23 downto 16)) +
148                            unsigned(data(15 downto 8));
149                 checksum_calculated <= std_logic_vector(temp_sum(7 downto 0));
150                 if (checksum_calculated = checksum_received) then
151                                 s_readyFlag <= '1';  -- Set ready flag (assume data is valid for now)
152                             else
153                                 s_readyFlag <= '0'; -- Data is invalid
154                             end if;
155
156                             currentState <= assign_output_state; -- Return to initial state for next cycle
157             when assign_output_state =>
158                 temperature_ten := s_temperature / 10 mod 10;
159                 temperature_unit := s_temperature mod 10;
160
161                 --common_bus(63 downto 56) <= std_logic_vector(temperature_ten + 48);
162                 --common_bus(55 downto 48) <= std_logic_vector(temperature_unit + 48);
163                 currentState <= zero_state;
164
```

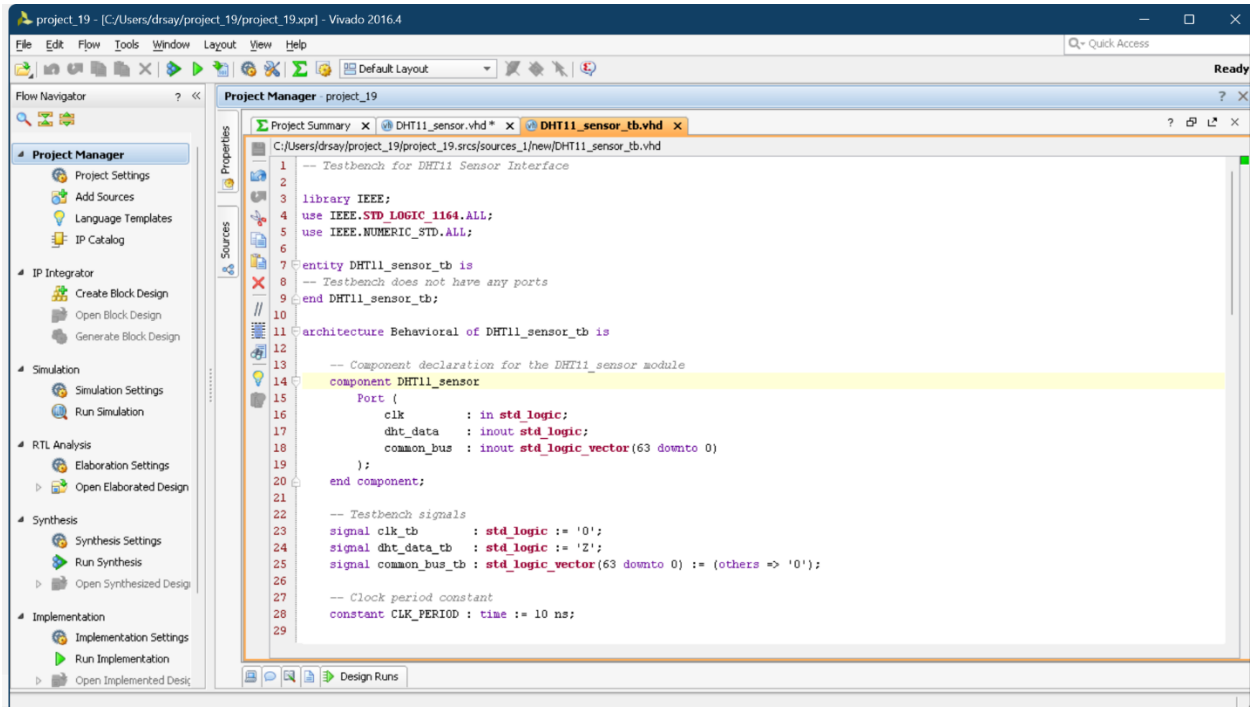


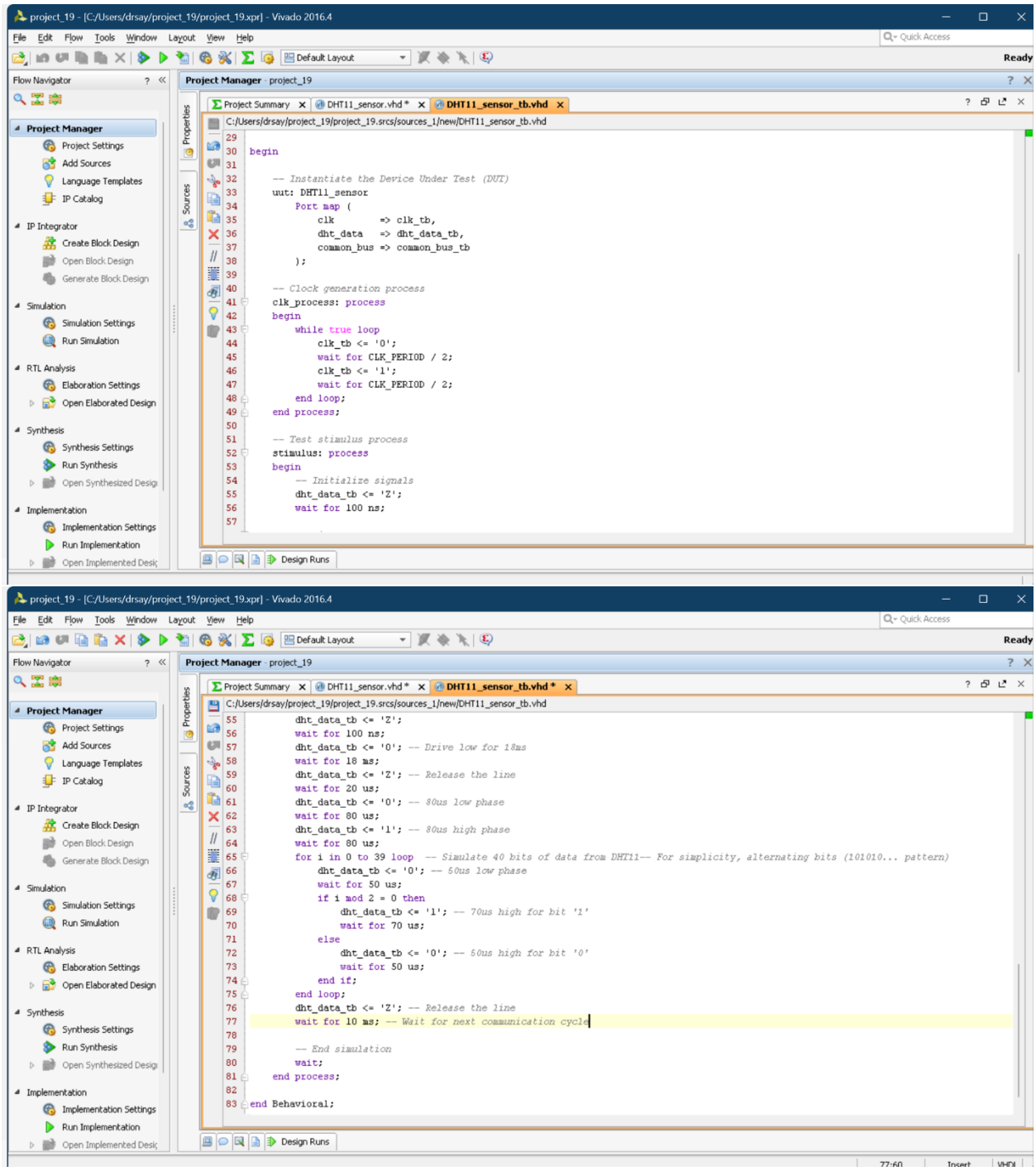

```
162                 --common_bus(55 downto 48) <= std_logic_vector(temperature_unit + 48);
163                 currentState <= zero_state;
164
165             -- Default case: Return to initial state
166             when others =>
167                 currentState <= zero_state;
168         end case;
169     end if;
170 end process;
171
172 --process(currentState)
173 --begin
174 --   case (currentState) is
175 --      when zero_state =>
176 --         if dht_data_dir = '1' then
177 --           dht_data <= dht_data_out;
178 --         else
179 --           dht_data <= 'Z';
180 --         end if;
181 --      when start_down_state =>
182 --          if dht_data_dir = '1' then
183 --              dht_data <= dht_data_out;
184 --          else
185 --              dht_data <= 'Z';
186 --          end if;
187 --       when others =>
188 --          temperature<="00000000";
189 --          temperature_dec<="00000000";
190 --          humidity<="00000000";
```




```
165             -- Default case: Return to initial state
166             when others =>
167                 currentState <= zero_state;
168         end case;
169     end if;
170 end process;
171
172 --process(currentState)
173 --begin
174 --   case (currentState) is
175 --      when zero_state =>
176 --         if dht_data_dir = '1' then
177 --           dht_data <= dht_data_out;
178 --         else
179 --           dht_data <= 'Z';
180 --         end if;
181 --      when start_down_state =>
182 --          if dht_data_dir = '1' then
183 --              dht_data <= dht_data_out;
184 --          else
185 --              dht_data <= 'Z';
186 --          end if;
187 --       when others =>
188 --          temperature<="00000000";
189 --          temperature_dec<="00000000";
190 --          humidity<="00000000";
191 --      end case;
192 -- end process;
193 end Behavioral;
```


project_20 - [C:/Users/drsay/project_20/project_20.xpr] - Vivado 2016.4
File Edit Flow Tools Window Layout View Run Help
Quick Access
Default Layout
Ready
Flow Navigator
Project Manager
Project Settings
Add Sources
Language Templates
IP Catalog
IP Integrator
Create Block Design
Open Block Design
Generate Block Design
Simulation
Simulation Settings
Run Simulation
RTL Analysis
Elaboration Settings
Open Elaborated Design
Synthesis
Synthesis Settings
Run Synthesis
Open Synthesized Design
Implementation
Implementation Settings
Run Implementation
Open Implemented Design
Behavioral Simulation - Functional - sim_1 - DHT11_sensor_tb
DHT11_sensor.vhd
DHT11_sensor_tb.vhd
Untitled 8
Name
Value
clk
dht_data
common_bus[63:0]
clk_period
10000 ps
701.164 ns
0 ns
200 ns
400 ns
600 ns
800 ns
Tcl Console


project_19 - [C:/Users/drsay/project_19/project_19.xpr] - Vivado 2016.4
File Edit Flow Tools Window Layout View Help
Quick Access
Default Layout
Ready
Flow Navigator
Project Manager
Project Settings
Add Sources
Language Templates
IP Catalog
IP Integrator
Create Block Design
Open Block Design
Generate Block Design
Simulation
Simulation Settings
Run Simulation
RTL Analysis
Elaboration Settings
Open Elaborated Design
Synthesis
Synthesis Settings
Run Synthesis
Open Synthesized Design
Implementation
Implementation Settings
Run Implementation
Open Implemented Design
Project Manager - project_19
Project Summary
DHT11_sensor.vhd *
DHT11_sensor_tb.vhd
C:/Users/drsay/project_19/project_19.srcs/sources_1/new/DHT11_sensor_tb.vhd
-- Testbench for DHT11 Sensor Interface
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity DHT11_sensor_tb is
-- Testbench does not have any ports
end DHT11_sensor_tb;
architecture Behavioral of DHT11_sensor_tb is
-- Component declaration for the DHT11_sensor module
component DHT11_sensor
Port (
clk : in std_logic;
dht_data : inout std_logic;
common_bus : inout std_logic_vector(63 downto 0)
);
end component;
-- Testbench signals
signal clk_tb : std_logic := '0';
signal dht_data_tb : std_logic := 'Z';
signal common_bus_tb : std_logic_vector(63 downto 0) := (others => '0');
-- Clock period constant
constant CLK_PERIOD : time := 10 ns;
Design Runs



Here is the drive that contains our codes and video:

https://drive.google.com/drive/u/0/folders/1XLDRS9gq8AhIFBP5GQdAdBeGjmAlp-tv